ITECH3102 DATA VISUALIZATION: COMPREHENSIVE LAB REPORT

Name: ClientX

Student ID: StudentIDX

Course: ITECH3102

Date: 17/09/2026

Labs 5 to 8.

Contents

CHAPTER 5: SIMPLE DATA VISUALIZATIONS 2

5.1 Project 1: Basic Bar Charts 2
5.2 Project 2: Dot Charts 2
5.3 Project 3: Line Charts 4
5.4 Project 4: Histogram 8
5.5 Project 5: Pie Chart 9

CHAPTER 6: COMPLEX DATA VISUALIZATIONS 10

6.1 Project 1: Using Aggregate Functions 10
6.2 Project 2: Using Calculated Fields 11
6.2.1 Loading the Dataset 11
6.2.2 Creating the Population Density Calculated Field 12
6.2.3 Creating the Filled Map 13
6.2.4 Editing the Geographic Location 14
6.2.5 Adding County Labels 15
6.2.6 Discussion 16
6.3 Project 3: Create a Logic Calculation 16
6.3.1 Case Discrimination 16
6.3.2 IF-THEN-ELSE Logic 18

CHAPTER 7: COMPLEX DATA VISUALIZATIONS (2) 22

7.1 Project 1: Visualize comparison using calculated fields 22
7.2 Project 2: Visualize baseball player batting averages with filters and rank 23
7.3 Project 3: Visualize home runs with team totals and percent of total 26

CHAPTER 8: COMPLEX DATA VISUALIZATIONS (3) 29

8.1 Project 1: Visualize trend 29
8.2 Project 2: Visualize forecasts 33
8.3 Project 3: Visualize cluster 35
8.4 Project 4: Use parameter control 38

Chapter 5: Simple Data Visualizations

5.1 Project 1: Basic Bar Charts

Procedure: The simple-data-boroughs.xlsx dataset was loaded into Tableau Public. To create a bar chart, the Population measure was dragged onto the Rows shelf and the Borough dimension onto the Columns shelf. The bars were then sorted in descending order using the Sort option. Finally, the Population field was dragged onto the Color mark, and the color was changed to green.

Result/Observation: The bar chart clearly displayed the population distribution across the five boroughs. Sorting the bars made comparisons immediate, and the green color provided a clear visual cue for magnitude. As discussed in Munzner's Chapter 7, arranging bars on a shared baseline enables accurate comparison of quantitative values.

Screenshots:

Figure 5.1: Initial bar chart of population by borough

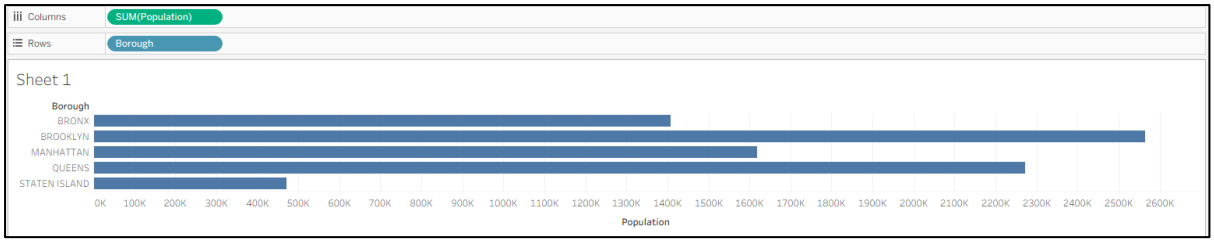


Figure 5.2: Bar chart sorted in descending order

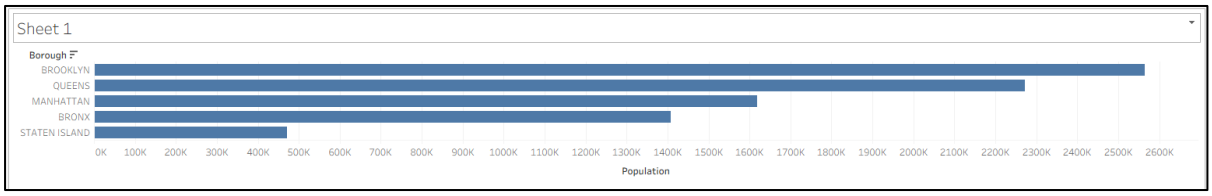
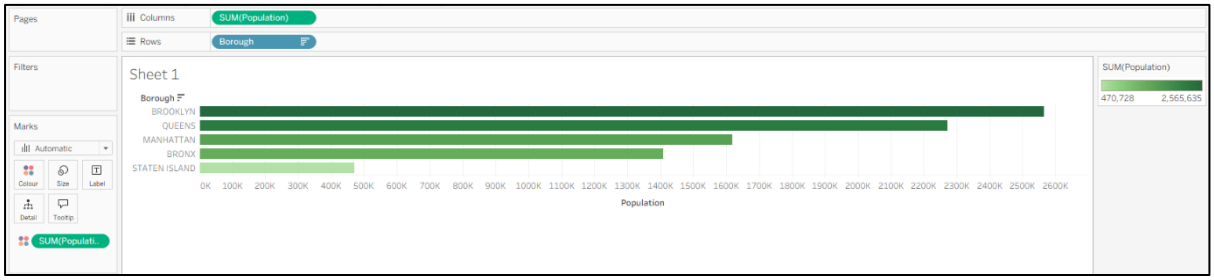


Figure 5.3: Bar chart with green color encoding



5.2 Project 2: Dot Charts

Procedure: The waste-collection.xlsx dataset was connected to Tableau. To build the dot chart, RefuseTonsCollected was placed on the Rows shelf and Borough on the Columns shelf. The Marks card type was then changed from Automatic to Shape, converting the bars into dots. To improve readability, grid lines were added from the Lines tab in the Format menu.

Result/Observation: The dot chart effectively displayed the total refuse collected by each borough. Unlike bar charts that use length, dot charts rely on position along a shared axis. Position is one of the most accurate perceptual channels for quantitative data, making this chart highly readable. Adding grid lines provided a reference frame that made it easier to estimate the exact value of each dot.

Screenshots:

Figure 5.4: Dot chart of refuse tons collected by borough

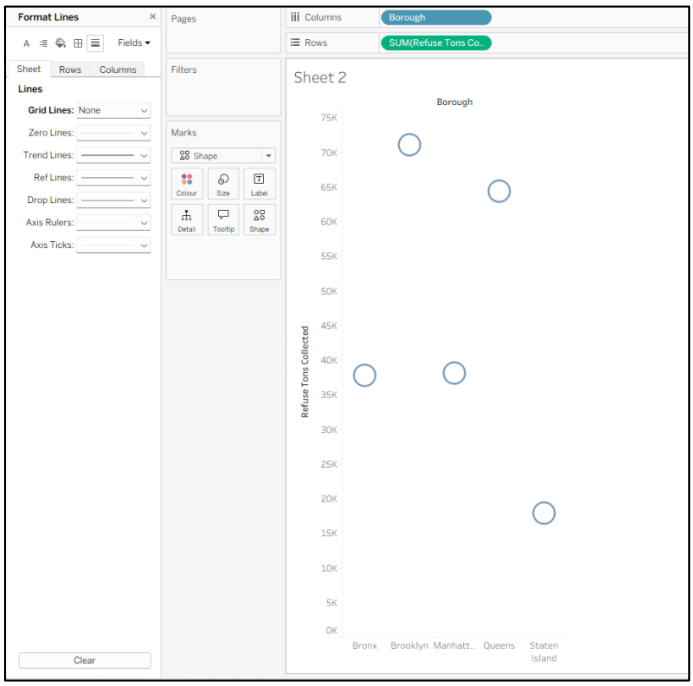
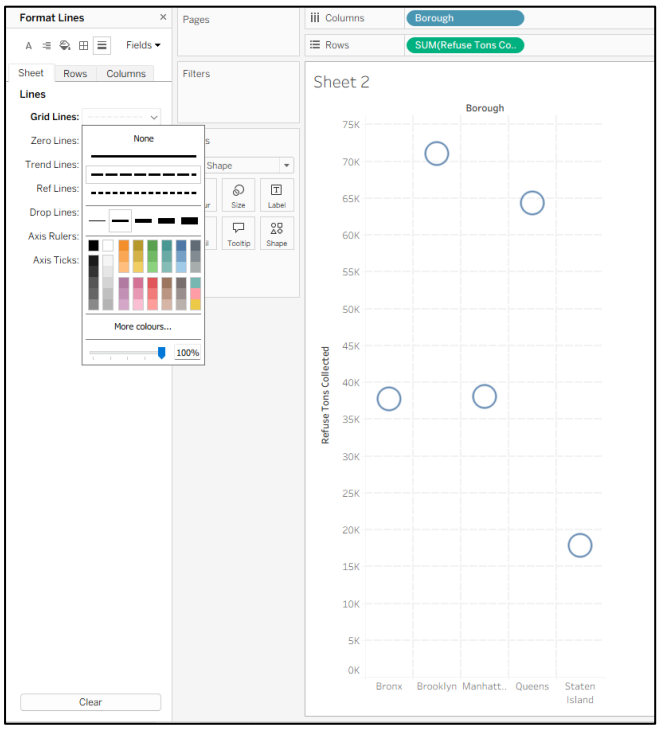


Figure 5.5: Dot chart with added grid lines



5.3 Project 3: Line Charts

Procedure: The order-data.xlsx dataset was loaded. A basic line chart was created by dragging Order Date to Columns and Sales to Rows. The YEAR(Order Date) field was expanded to Quarter level. To compare quarters across years, YEAR(Order Date) was moved to Color. Continuous line charts were explored using the Month date part. The Path option on the Marks card was used to demonstrate Step and Jump line styles, and markers were added through the Color menu.

Result/Observation: The line chart showed a clear upward trend in sales from 2018 to 2021. Drilling down to quarter level revealed seasonal patterns. Moving Year to Color produced separate colored lines for easy comparison. The Step chart clearly highlighted sudden changes between points, while the Jump chart emphasised gaps in the data. These options demonstrate how line style can communicate different types of temporal change.

Screenshots:

Figure 5.6: Basic line chart of sales over time

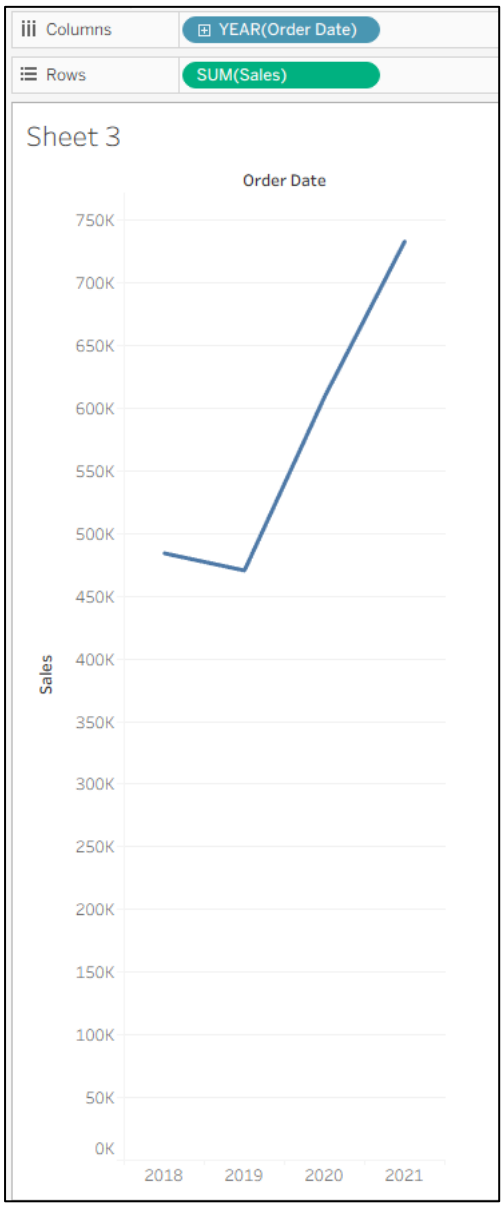


Figure 5.7: Line chart drilled down to quarter level

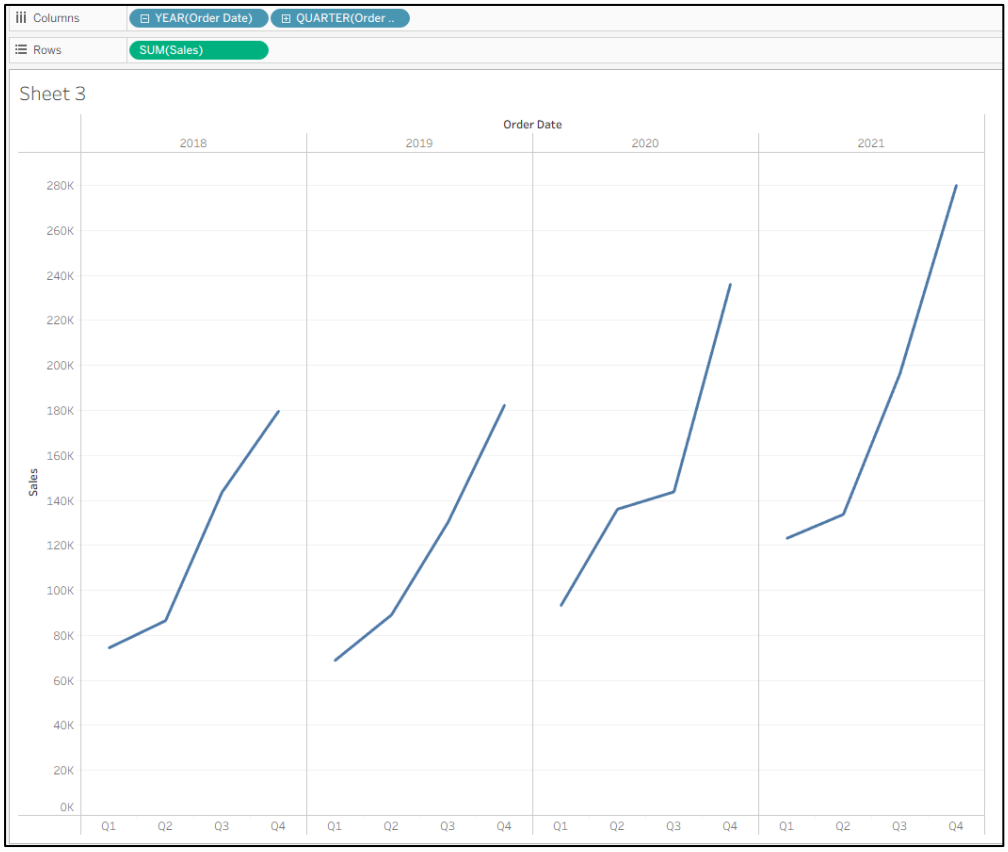


Figure 5.8: Line chart with Year in Color

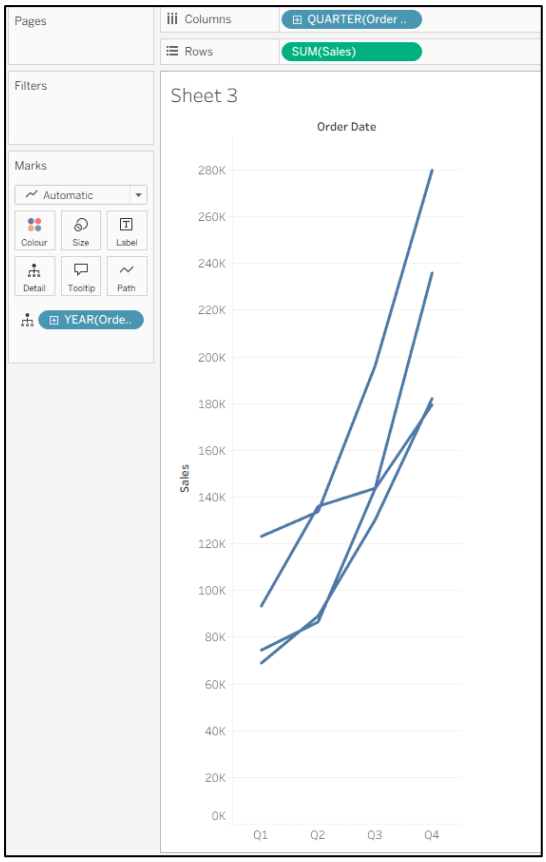


Figure 5.9: Continuous line chart by month

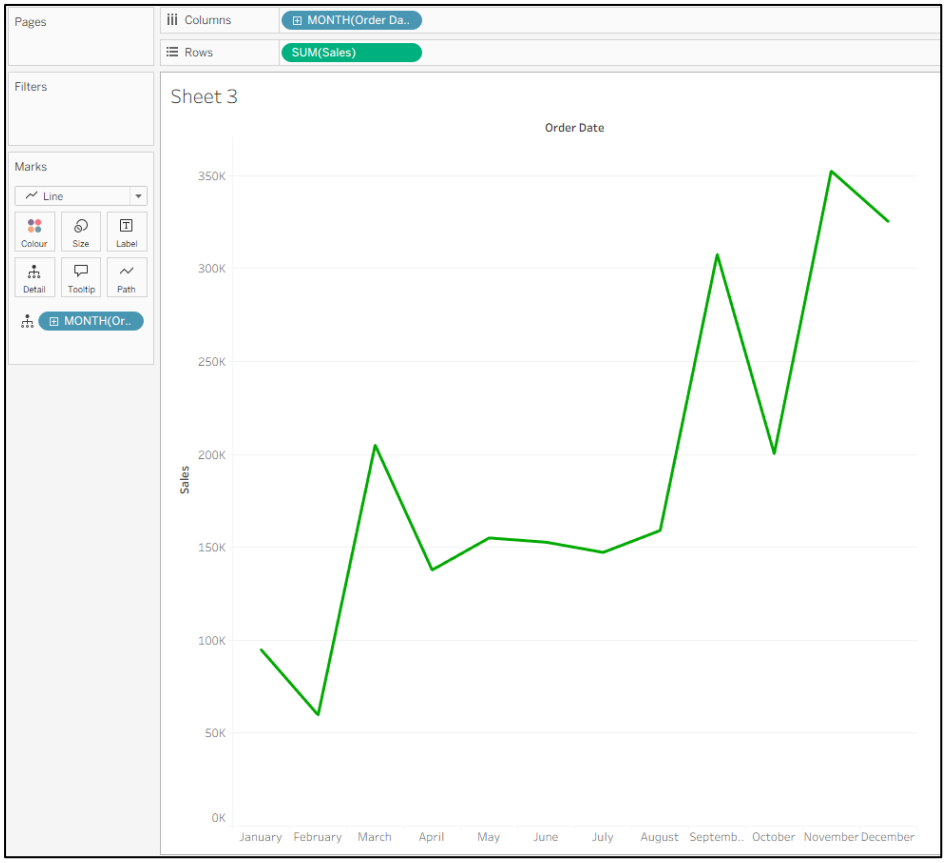


Figure 5.10: Continuous line chart with markers

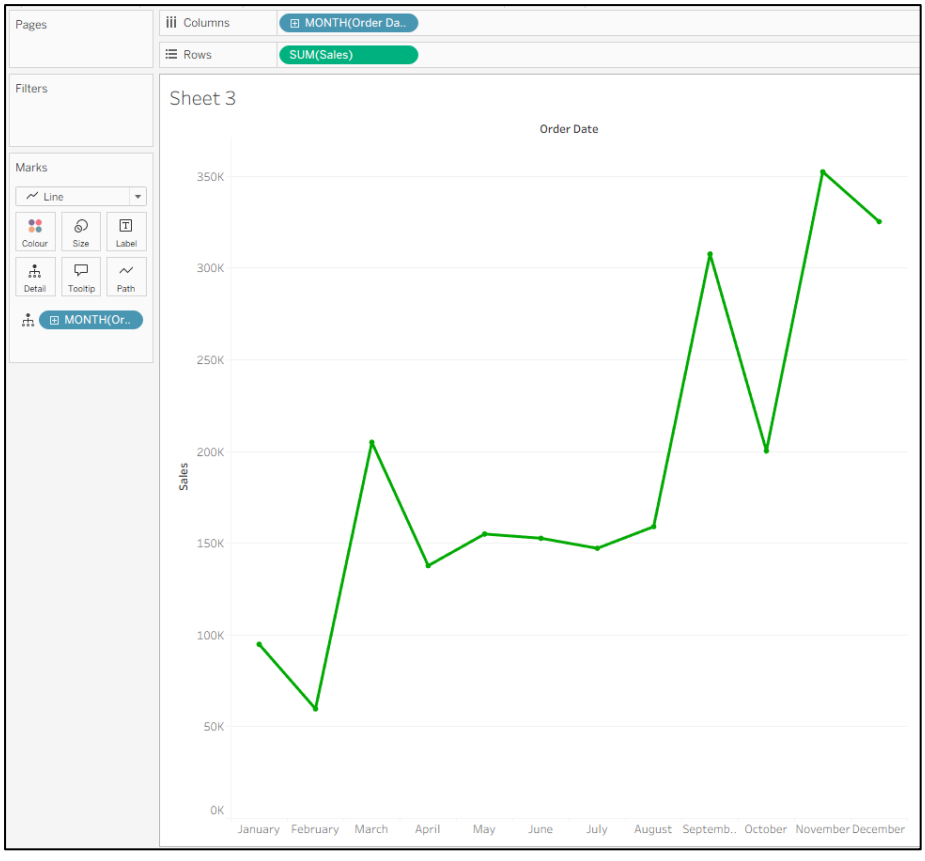


Figure 5.11: Step line chart

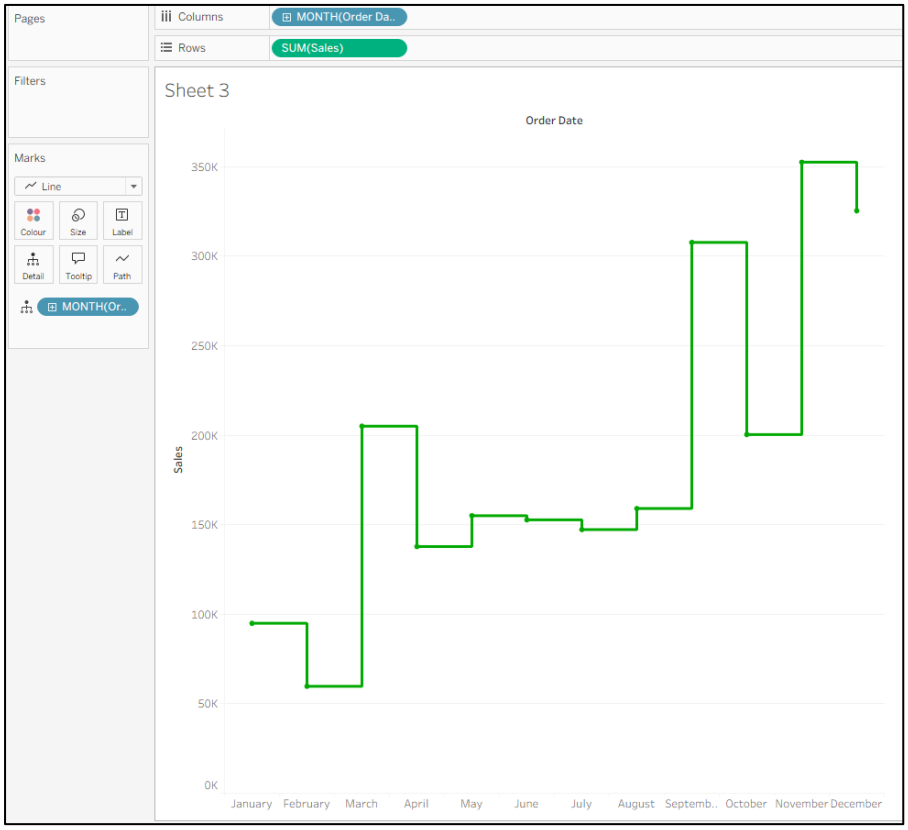
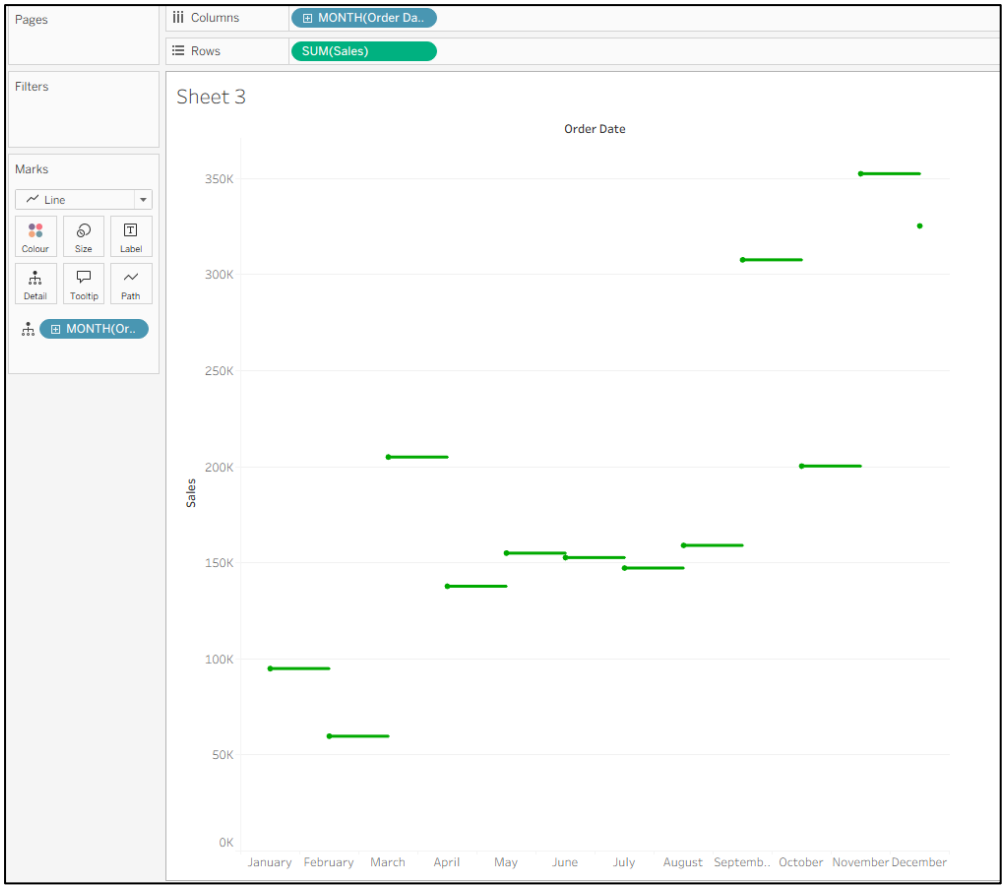


Figure 5.12: Jump line chart



5.4 Project 4: Histogram

Procedure: The waste-collection.xlsx dataset was used. The RefuseTonsCollected field was selected, and the Show Me card was used to select the Histogram chart type. Tableau automatically generated a RefuseTonsCollected (bin) dimension. To modify the level of detail, the bin field was right-clicked, Edit was selected, and the bin size was changed to 100.

Result/Observation: The histogram provided a clear picture of the frequency distribution. With the default bin size, the distribution appeared as a broad bell shape. After reducing the bin size to 100, the chart revealed more granular peaks. This demonstrated how bin size controls the trade-off between detail and smoothness in a histogram.

Screenshots:

Figure 5.13: Initial histogram of refuse tons collected

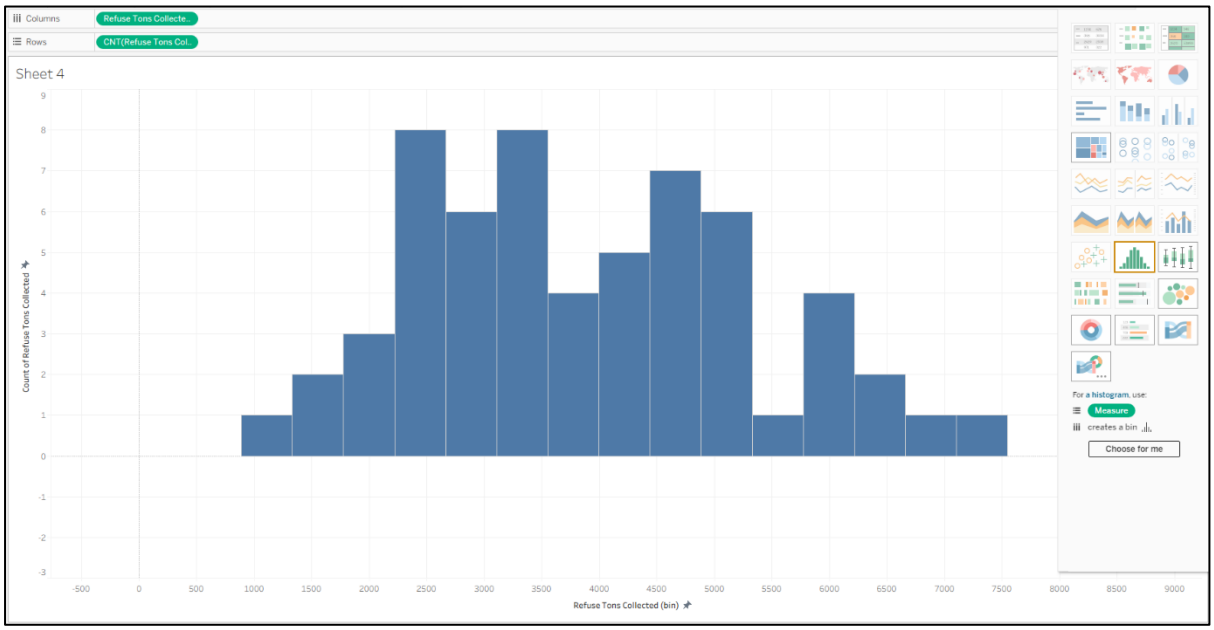
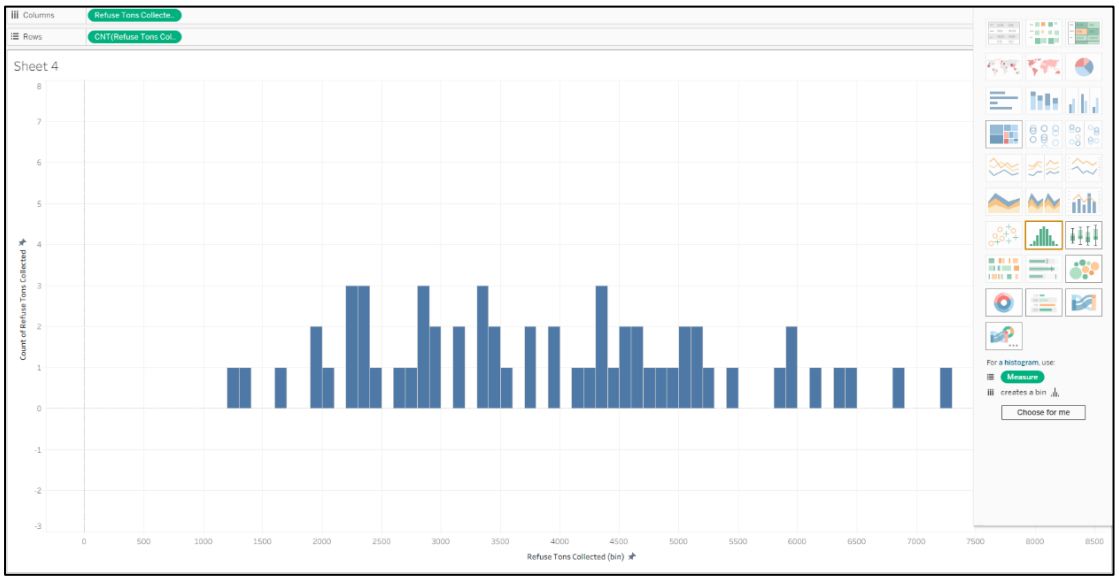


Figure 5.14: Histogram with bin size changed to 100



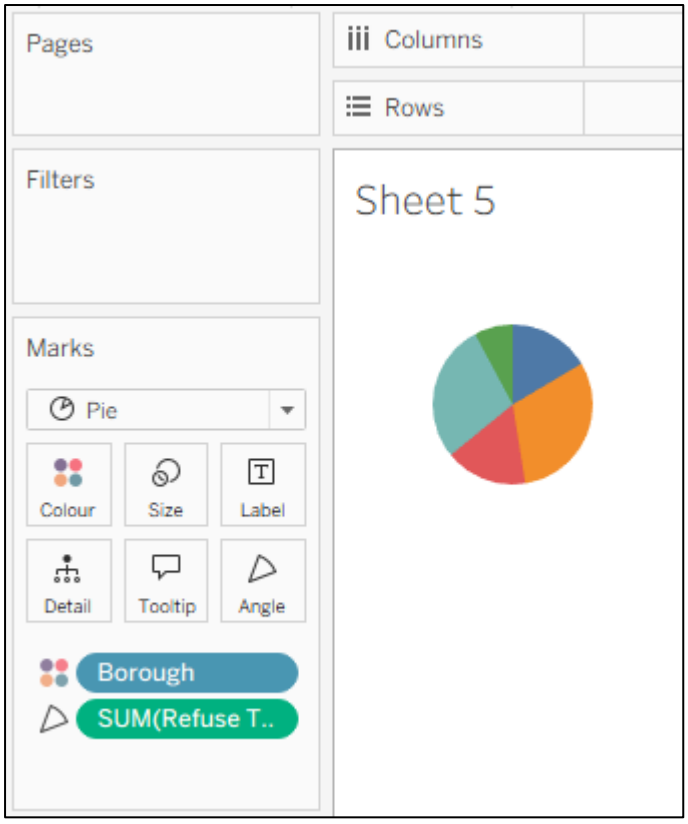
5.5 Project 5: Pie Chart

Procedure: Using the waste-collection.xlsx dataset, a pie chart was created by dragging Borough onto the Color mark and RefuseTonsCollected onto the Angle mark. The Marks type was set to Pie.

Result/Observation: The pie chart showed the relative share of total refuse collected by each borough. While pie charts make precise comparison difficult, they are effective for showing part-to-whole relationships at a glance. As noted in "Storytelling with Data," pie charts should be used sparingly, but they work well for simple proportional breakdowns.

Screenshots:

Figure 5.15: Pie chart of refuse collected by borough



Chapter 6: Complex Data Visualizations

6.1 Project 1: Using Aggregate Functions

Procedure: The order-information.xls dataset was loaded into Tableau Public through the Microsoft Excel connector, and the Orders worksheet was dragged onto the data model canvas. After selecting the Sheet 1 tab, the Sales measure was dragged from the Measures section of the Data pane onto the Rows shelf, producing a vertical bar chart in which a single bar represented the summed sales across all 9,994 records. The default SUM(Sales) pill on the Rows shelf was then clicked, the Measure (Sum) submenu was expanded, and the Median aggregation was selected to replace the default sum. The workbook was saved and both states of the chart were exported as screenshots for the report.

Result/Observation: The initial bar chart displayed a single tall bar reaching approximately 2.3 million, which represented the aggregated sales value across the entire dataset. After switching to the MEDIAN(Sales) aggregation, the bar height dropped to approximately 55, reflecting the median value of the individual sales transactions rather than their total. This change illustrates the fundamental distinction between an aggregate function that accumulates all values and one that returns a central value; as Munzner emphasises in *Visualization Analysis and Design*, the choice of aggregation is a derived-data decision that directly shapes what pattern a chart reveals, because the same underlying attribute can encode very different meanings depending on how it is summarised. The result also demonstrates that Tableau applies an aggregation function to every measure by default, and that the aggregation type is not merely a formatting choice but a semantic one.

Screenshots:

Figure 6.1: Bar chart of sales aggregated by Sum

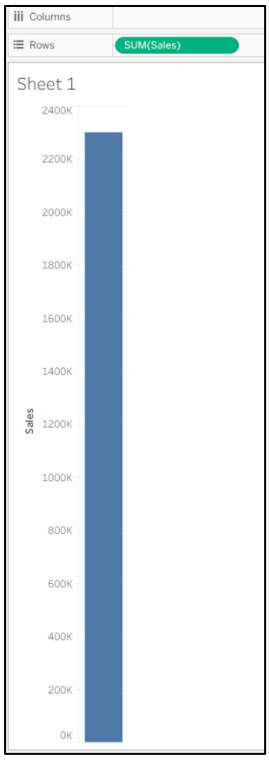
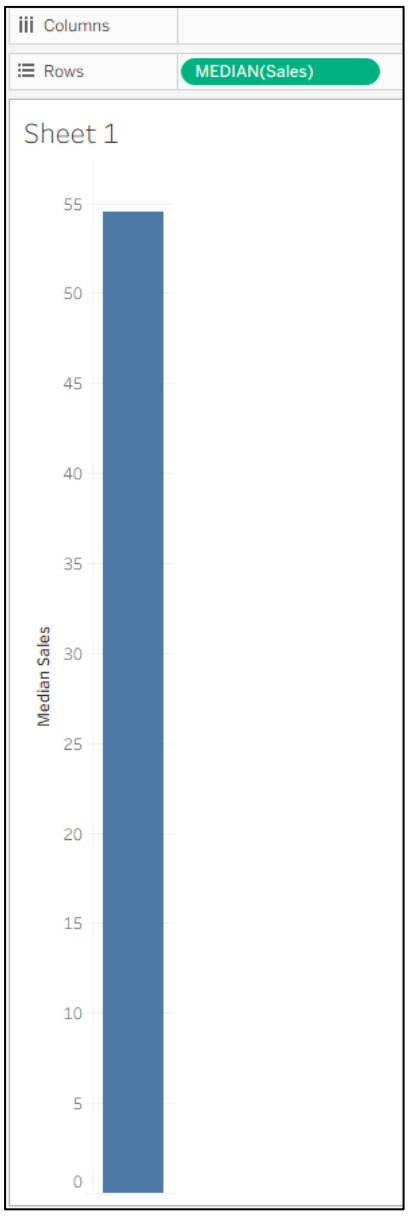


Figure 6.2: Bar chart of sales aggregated by Median



6.2 Project 2: Using Calculated Fields

6.2.1 Loading the Dataset

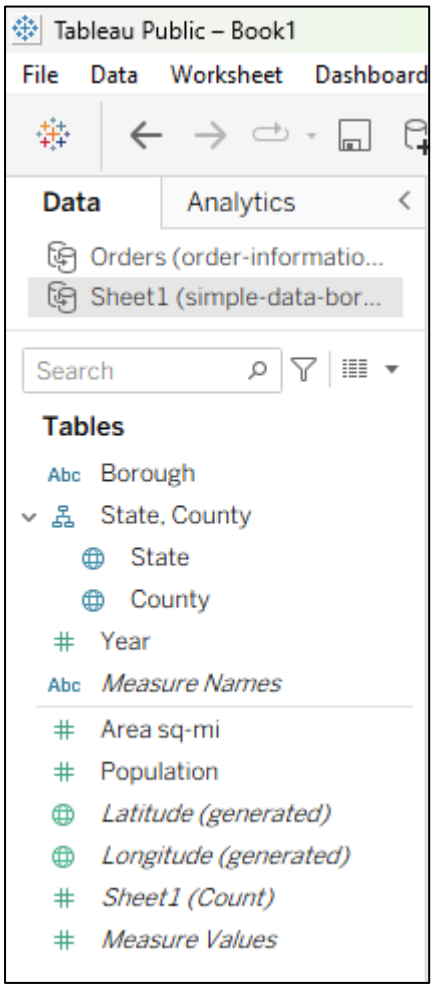
Procedure: A second data source was added to the existing workbook by selecting Data -> New Data Source -> Microsoft Excel and loading the simple-data-boroughs.xlsx file. The Sheet1 table was dragged onto the data model canvas, and the new data source was activated from the data source dropdown at the top of the Data pane. The dataset contained six fields and five rows, describing the five boroughs of New York City together with their county names, populations, land areas, and reference year.

Result/Observation:
Because the workbook already contained the order-information dataset, adding simple-data-

boroughs as a separate data source allowed the two datasets to be explored independently within the same workbook. This separation is consistent with the principle that data sources should be treated as distinct analytical contexts; as Loth notes in *Visual Analytics with Tableau*, Tableau encourages keeping unrelated datasets in separate connections so that aggregation levels and joins do not accidentally corrupt the meaning of a view.

Screenshots:

Figure 6.3: Second data source loaded into the workbook



6.2.2 Creating the Population Density Calculated Field

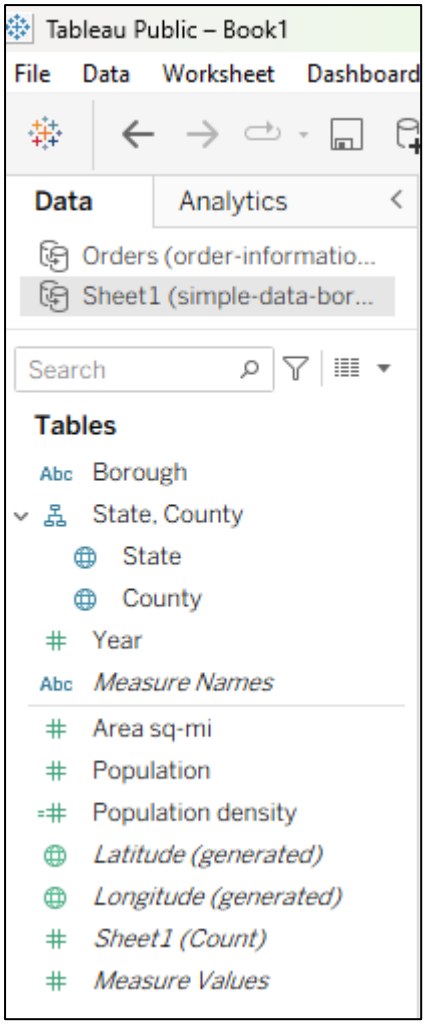
Procedure: A calculated field was created by right-clicking inside the Data pane and selecting Create -> Calculated Field. The field was named Population density and the formula [Population]/[Area sq-mi] was entered into the calculation editor, after which the calculation was confirmed as valid and the OK button was clicked. The new measure appeared in the Measures area of the Data pane alongside Population and Area sq-mi.

Result/Observation: The calculated field demonstrated that new measures not present in the raw data source can be derived arithmetically from existing measures. As discussed in Munzner's *Visualization Analysis and Design*, derived attributes are essential for exposing

relationships that would otherwise remain hidden, and population density is a classic example because neither Population nor Area sq-mi alone conveys how crowded a place is. The division of one measure by another produced a ratio whose value is only meaningful in context — a high population is not the same as a high population density.

Screenshots:

Figure 6.4: Calculated field definition for Population density



6.2.3 Creating the Filled Map

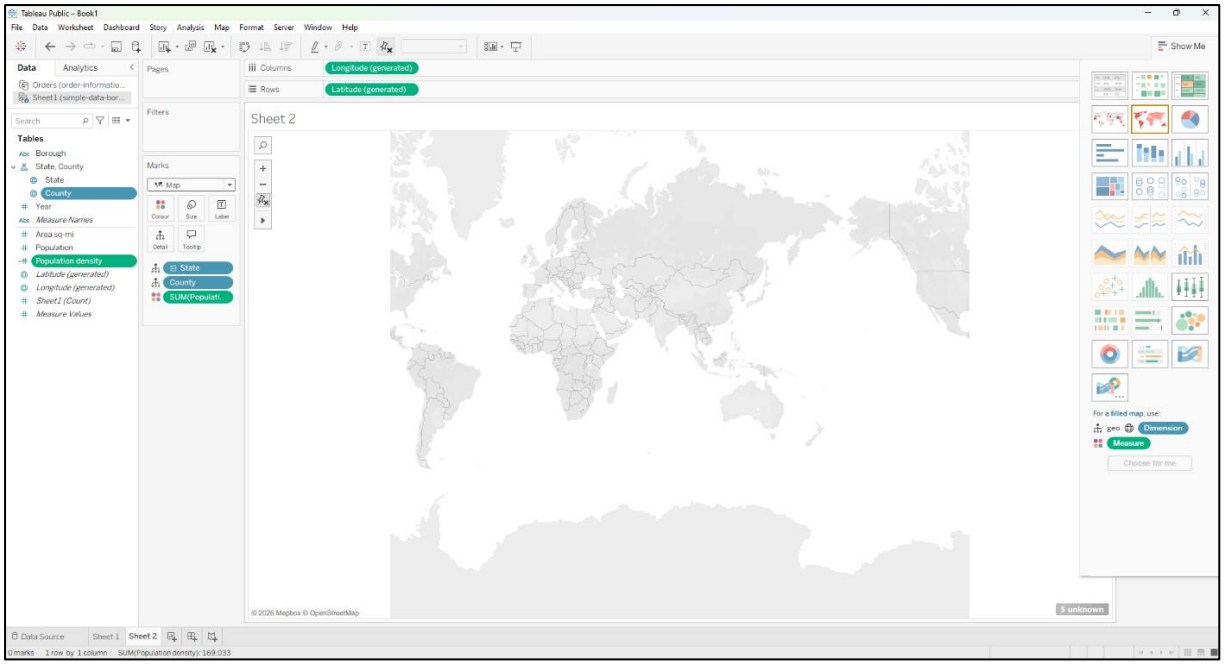
Procedure: The County dimension and the Population density measure were selected together with the Control key held down, and the Filled Map chart type was selected from the Show Me panel. Tableau automatically placed Longitude (generated) on the Columns shelf and Latitude (generated) on the Rows shelf, and added State and County to the Detail shelf of the Marks card while SUM(Population density) was added to the Colour shelf. The resulting view rendered a world map with the five geographic records initially marked as unknown.

Result/Observation: The filled map encoded population density as a colour gradient, with darker shades representing higher density. This use of colour to encode a quantitative measure

follows what Ware describes in *Information Visualization: Perception for Design* as a preattentive visual channel — colour intensity is processed rapidly by the visual system, allowing the viewer to compare the five boroughs almost instantaneously. At this stage, however, the five records were rendered as "5 unknown" because Tableau had defaulted to interpreting the County field against Australia's geographic hierarchy, illustrating that automated geographic role assignment is not always correct.

Screenshots:

Figure 6.5: Initial filled map with unrecognised geographic records



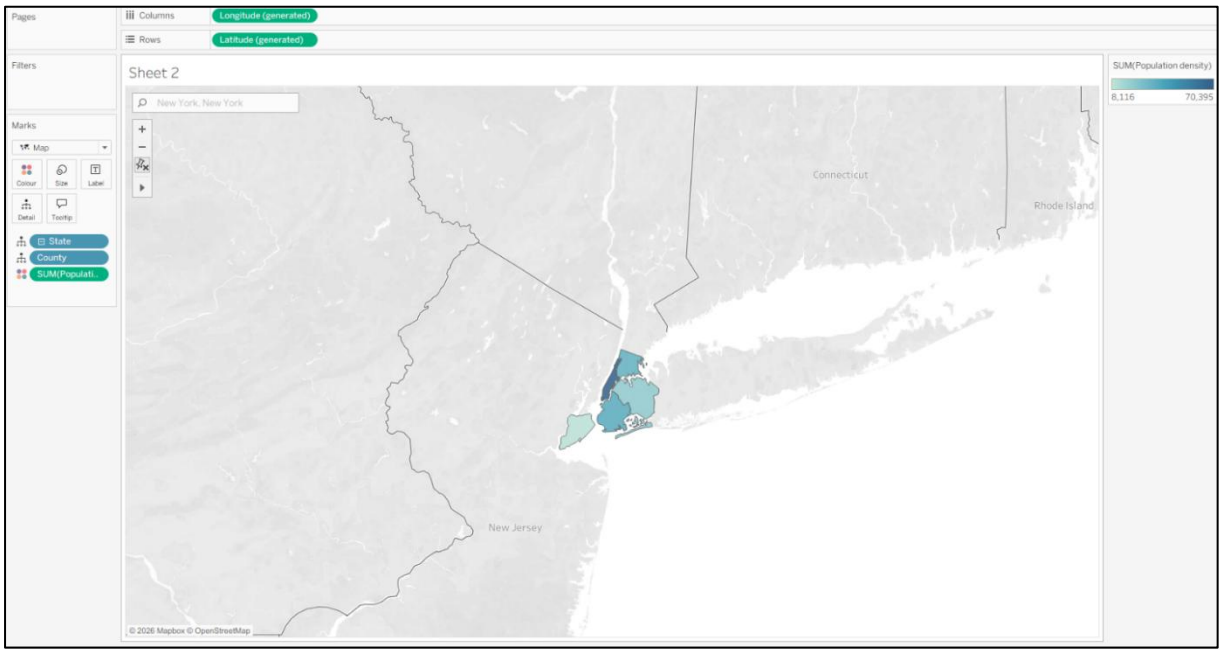
6.2.4 Editing the Geographic Location

Procedure: The 5 unknown indicator at the bottom-right corner of the map was clicked, and Edit Locations... was selected from the resulting dialog. In the Edit Locations window, the Country/Region dropdown was changed from Australia to USA, which caused the New York value under the County matching list to resolve against the correct geographic hierarchy. The OK button was then clicked, and Tableau automatically zoomed the map to the New York metropolitan area, where the five boroughs were rendered as shaded polygons.

Result/Observation: The correction of the geographic hierarchy shows that geographic roles in Tableau depend on both the field's data type and the country context; changing only the country resolved all five unrecognised records without any manual mapping of individual county names. Once resolved, the map displayed the five boroughs of New York City with a continuous colour gradient from light blue to dark blue, revealing that Manhattan (New York County) has by far the highest population density, while Staten Island (Richmond County) has the lowest. This outcome reinforces Munzner's argument that a good visual encoding makes comparisons effortless; colour intensity does the work of a numeric comparison without requiring the viewer to read any numbers.

Screenshots:

Figure 6.6: Filled map after correcting the geographic role to USA



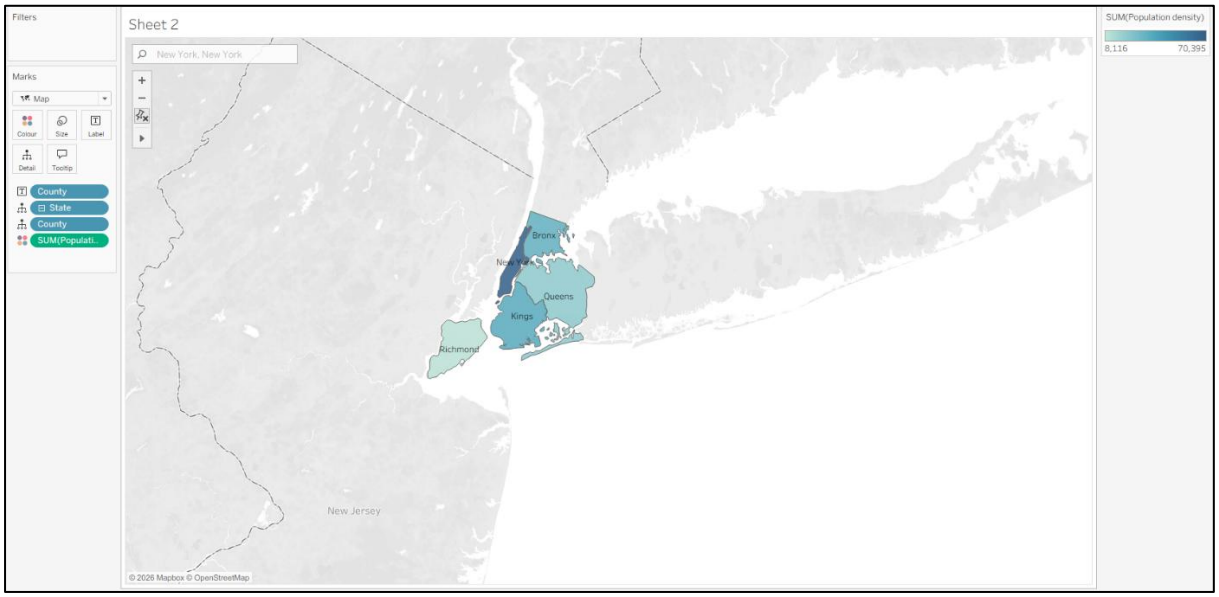
6.2.5 Adding County Labels

Procedure: The County dimension was dragged from the Data pane onto the Label button in the Marks card, which added a County pill to the Label shelf. The labels were then rendered directly on top of each borough polygon, showing the names New York, Bronx, Kings, Queens, and Richmond.

Result/Observation: Adding labels transformed the map from a purely visual comparison into a chart that can also be read numerically and textually. As Knaflic argues in *Storytelling with Data*, direct labelling of data points reduces the need for a separate legend and shortens the perceptual distance between the viewer and the data. The county names visible on the map also clarify that the underlying geography of New York City is organised by county rather than by borough name, which explains why the map shows Kings instead of Brooklyn and Richmond instead of Staten Island.

Screenshots:

Figure 6.7: Filled map with county labels



6.2.6 Discussion

Result/Observation: Based on the filled map, **New York County (Manhattan) has the highest population density**, with approximately 70,395 people per square mile, followed by Kings County (Brooklyn) at approximately 36,135, Bronx County at approximately 33,535, Queens County at approximately 20,851, and Richmond County (Staten Island) at approximately 8,116. The reason Manhattan has the highest density is that it concentrates a population of about 1.62 million into only 23 square miles of land, whereas Queens, for example, has a slightly larger population but occupies 109 square miles. This comparison illustrates the central insight behind the calculated field: population alone does not determine density, and only the ratio between population and land area reveals how concentrated the residents of each county actually are.

6.3 Project 3: Create a Logic Calculation

6.3.1 Case Discrimination

Procedure: A new worksheet was created and the simple-data-boroughs data source was activated. A calculated field named Borough Code was created from the Data pane by selecting Create -> Calculated Field, and a CASE statement was entered that mapped each borough name to a numeric code, with an ELSE clause returning the text not defined for any unmatched value. After the calculation was confirmed as valid, the Borough and Borough Code dimensions were dragged from the Data pane onto the Rows shelf, producing a two-column text table listing each borough alongside its assigned code.

Result/Observation: The CASE statement demonstrated how categorical values in a dimension can be translated into derived identifiers without any changes to the underlying data source. As Munzner notes in *Visualization Analysis and Design*, such logic calculations belong to the category of derived data and often serve as the intermediate step that enables later visual encodings, such as sorting, filtering, or grouping. The resulting table showed the five boroughs in alphabetical order with codes 2, 3, 1, 4, and 5 respectively, and the code was recorded as a discrete dimension rather than a measure because it holds no arithmetic meaning.

Screenshots:

Figure 6.8: Calculated field definition for Borough Code

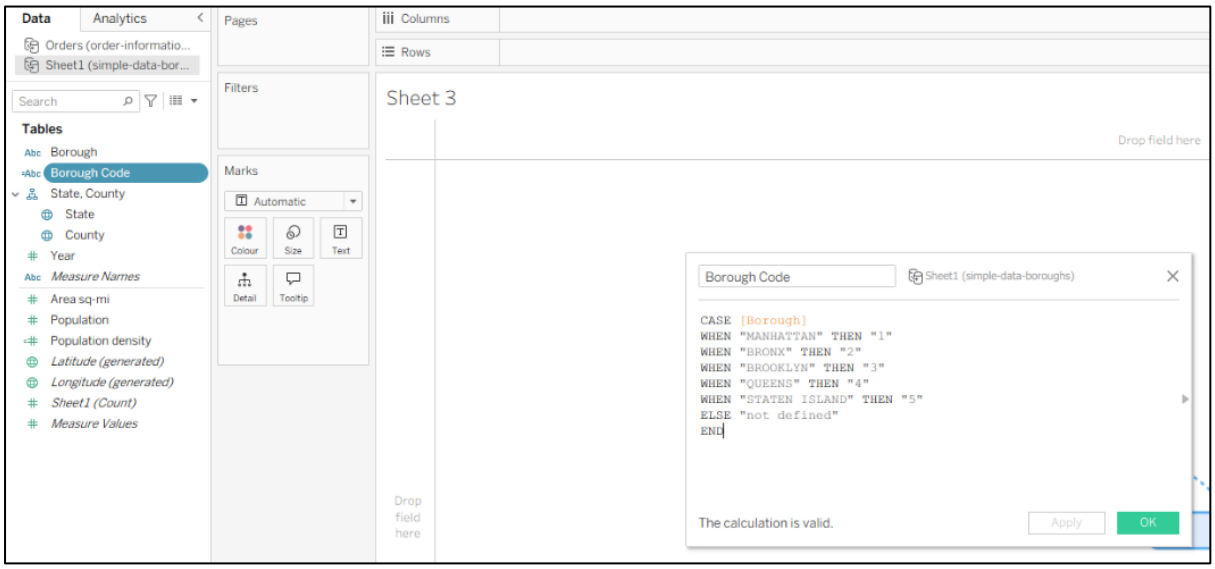
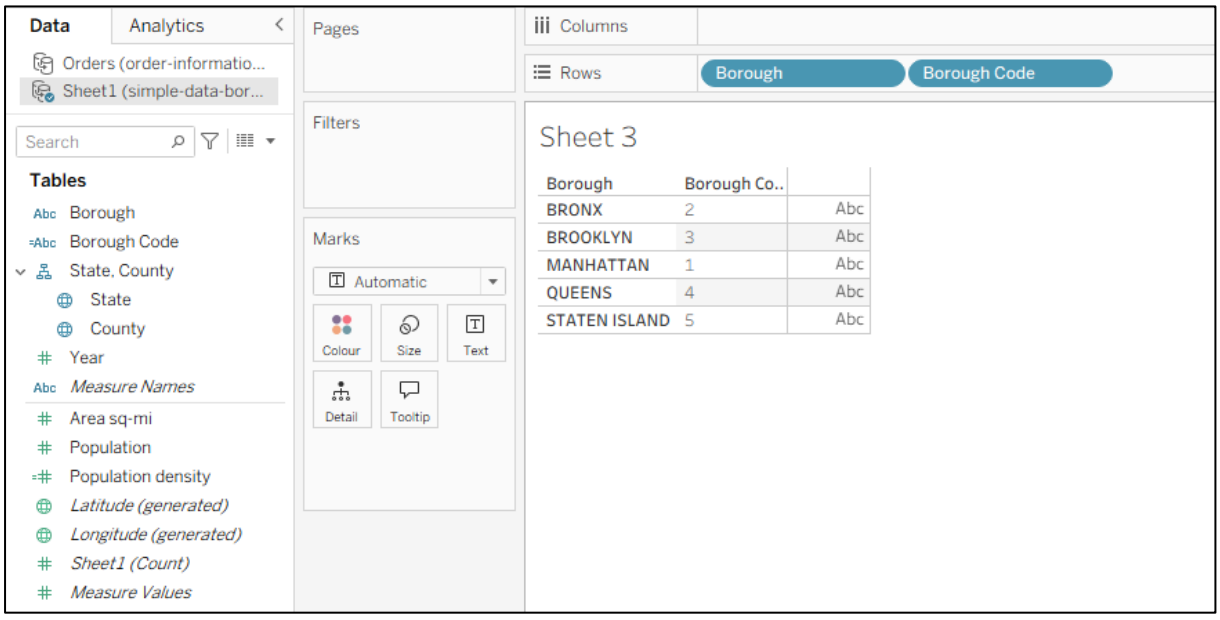


Figure 6.9: Borough and Borough Code displayed on the Rows shelf



6.3.2 IF-THEN-ELSE Logic

Procedure: A second worksheet was created and the order-information data source was activated. A vertical bar chart was produced by dragging Sub-Category onto the Columns shelf and Sales onto the Rows shelf, after which a calculated field named is sales <100K? was created from the Data pane. The initial formula evaluated the transactional Sales field with an IF ... THEN ... ELSE ... END statement and returned either Below 100K or Above 100K. This field was dragged to the Colour shelf of the Marks card, but the bars did not change colour. The calculated field was then edited so that the transactional Sales was replaced by the aggregate SUM([Sales]), and after the modification the bars were coloured in two shades. A second calculated field named sales with 3 color was subsequently created using an IF ... ELSEIF ... ELSE ... END structure that classified each sub-category into one of three bands: <100K, 100K-200K, and >200K. The previous colour pill was removed, the new sales with 3 color field was dragged to the Colour shelf, and the chart was rendered with three distinct colours.

Result/Observation: The first version of the calculated field evaluated Sales at the transaction level, but the worksheet displayed the aggregated SUM(Sales); because every individual transaction recorded a value below 100,000, the field returned the same value for every row and the colour encoding was effectively a single category. Replacing [Sales] with SUM([Sales]) aligned the level of detail in the calculation with the level of detail in the view, at which point each bar carried a meaningful classification. This is a well-known source of error in Tableau and is discussed by Loth in *Visual Analytics with Tableau*, where he distinguishes between a measure used at the transactional level and the same measure used as an aggregate. The three-colour version then extended the classification into a range-based encoding, which, as Ware argues in *Information Visualization: Perception for Design*, is a natural fit for colour because the visual system is highly sensitive to hue differences across small numbers of categories. The final chart made the distribution of sub-categories across the three sales bands immediately visible.

Screenshots:

Figure 6.10: Initial bar chart of sales by Sub-Category

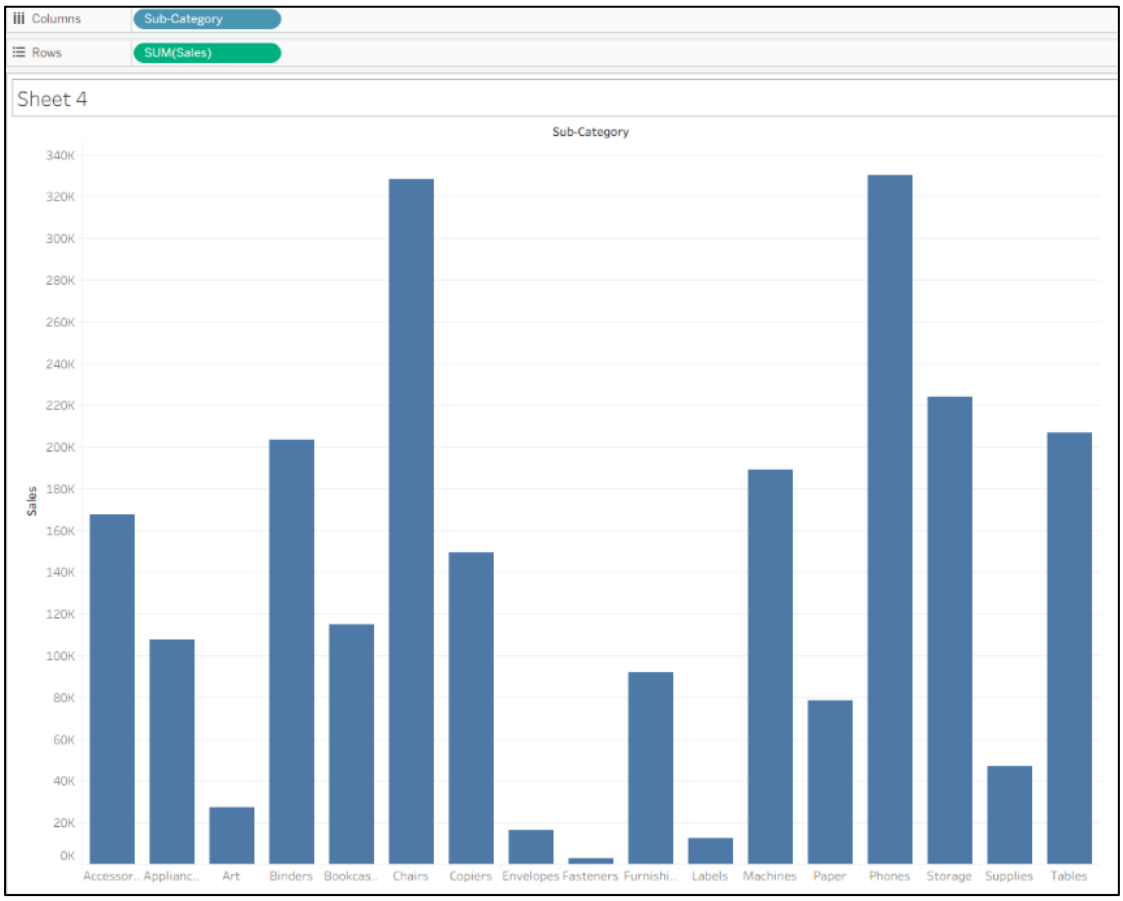


Figure 6.11: Calculated field definition for is sales <100K?

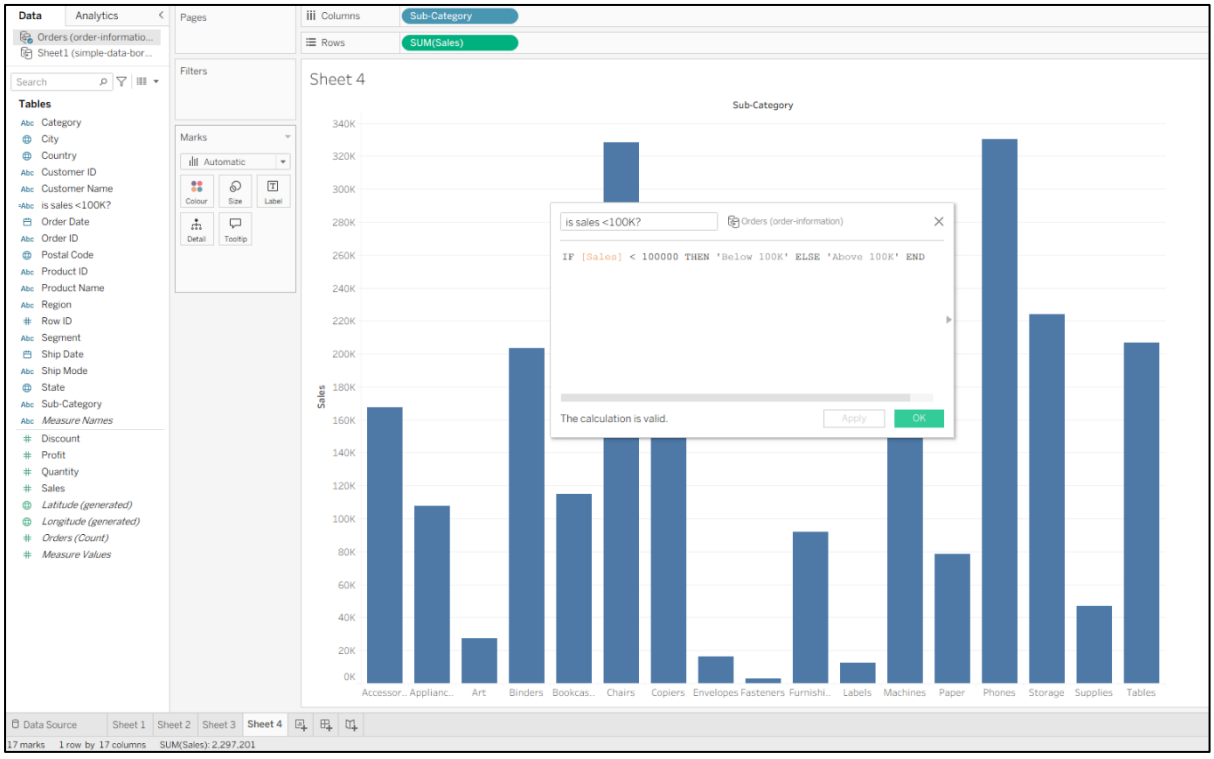


Figure 6.12: Unchanged chart after applying the transactional-level calculated field

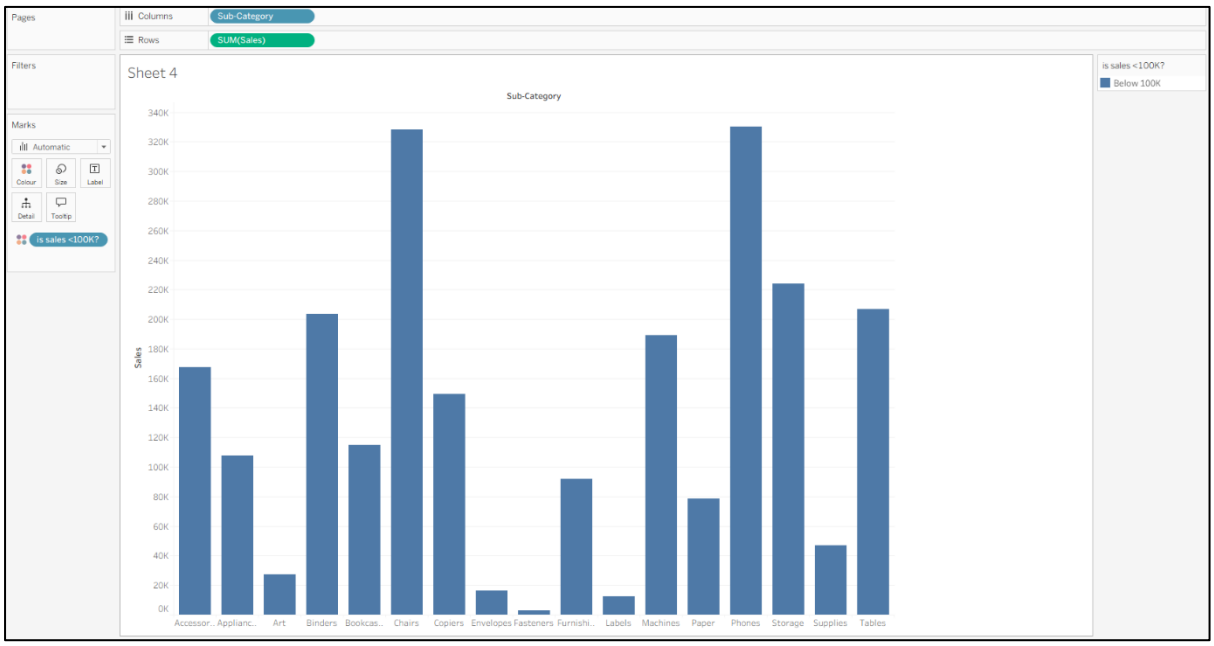


Figure 6.13: Two-colour chart after modifying the calculated field to use SUM([Sales])

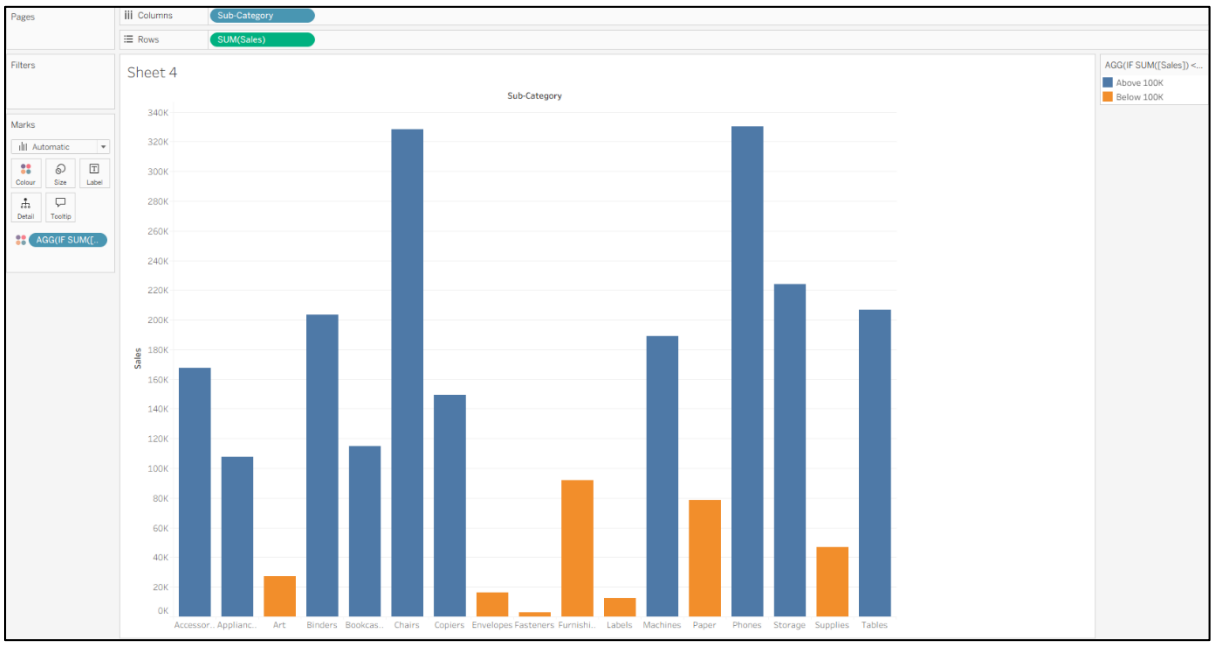
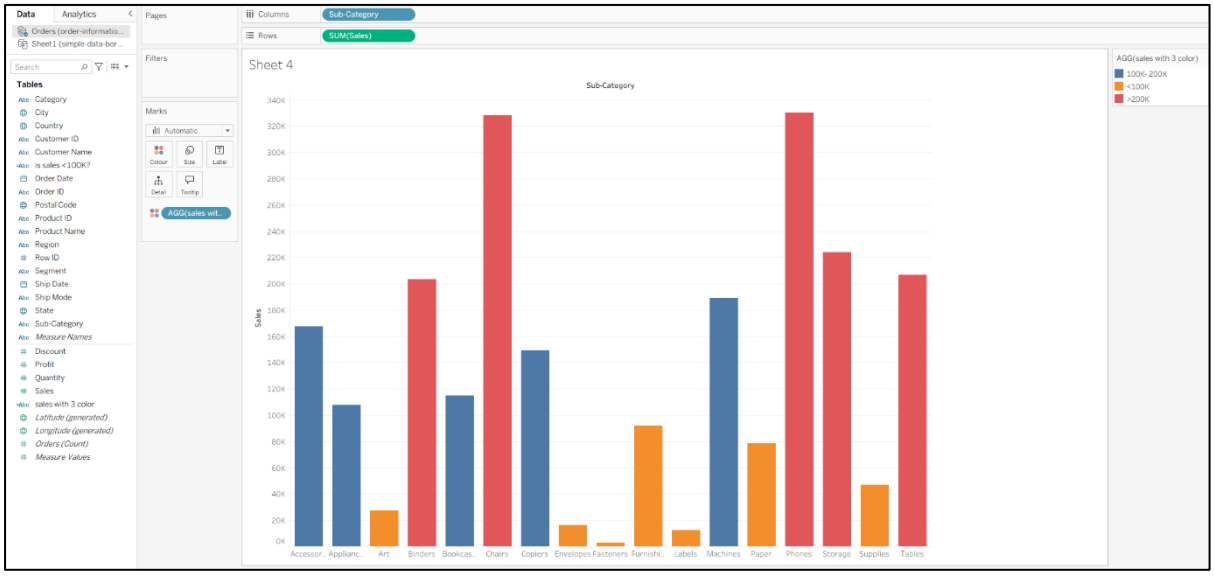


Figure 6.14: Three-colour chart using the sales with 3 color calculated field



Chapter 7: Complex data visualizations (2)

7.1 Project 1: Visualize comparison using calculated fields

Procedure: The waste-collection.xlsx dataset was loaded into Tableau Public through the Microsoft Excel connector, and the waste-collection sheet was dragged onto the data model canvas. After selecting the Sheet 1 tab, two calculated fields were created from the Data pane. The first field, RecyclableTonsCollected, was defined as the sum of [Paper Tons Collected] and [MGP Tons Collected], while the second field, Recycle to Refuse Ratio, was defined as SUM([RecyclableTonsCollected]) / SUM([Refuse Tons Collected]). The Borough dimension was then dragged onto the Rows shelf, the Recycle to Refuse Ratio measure was placed on the Columns shelf, and Borough was applied to the Colour shelf of the Marks card, producing a coloured horizontal bar chart that was sorted in descending order. Finally, the Borough field was moved to the Columns shelf, the Community District field was dragged onto the Rows shelf in discrete form, and Recycle to Refuse Ratio was added to the Label shelf with its number format changed to two decimal places.

Result/Observation: The high-level bar chart revealed that Manhattan had the highest recycle-to-refuse ratio at approximately 0.26, while the Bronx recorded the lowest value at approximately 0.13. When the analysis was extended to the community district level, the grid of bar charts demonstrated that certain districts within the Bronx achieved ratios higher than several districts in Manhattan. This pattern illustrates the importance of aggregation level in visual analytics; as Munzner emphasises in *Visualization Analysis and Design*, the choice of granularity directly determines which comparative patterns become visible. The colour encoding of boroughs supports preattentive grouping, which, as Ware argues in *Information Visualization: Perception for Design*, allows viewers to compare segments of the same borough across the grid without conscious effort. The final two-decimal label format also reflects the decluttering principle noted by Knaflic in *Storytelling with Data*, where unnecessary precision reduces the legibility of a chart.

Screenshots:

Figure 7.1: waste-collection dataset loaded into Tableau's data source page

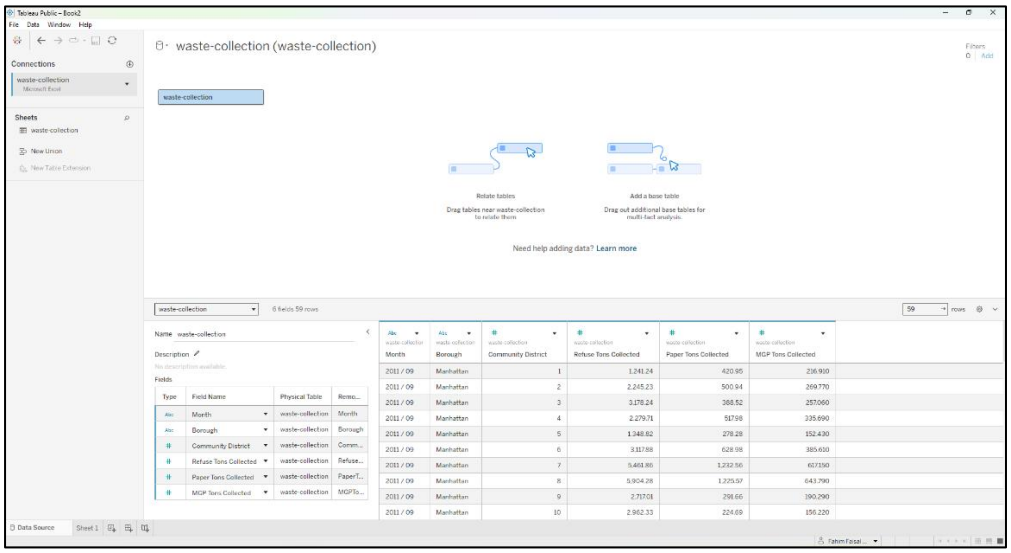


Figure 7.2: Recycle-to-refuse ratio for each borough at the high-level view

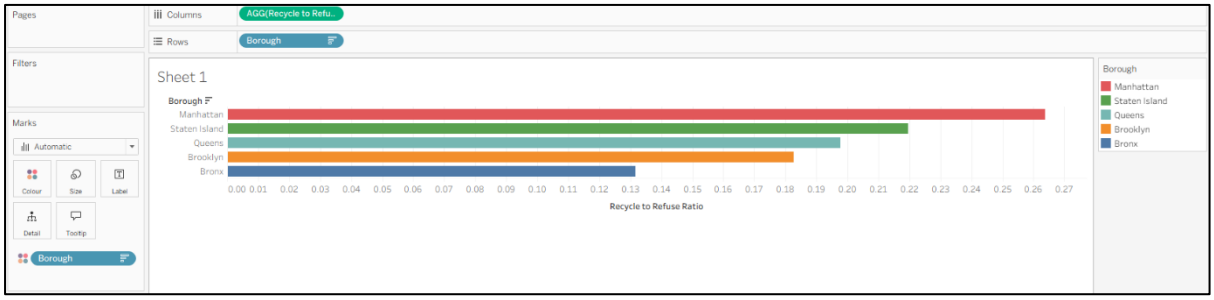


Figure 7.3: Community district ratio grid without labels



Figure 7.4: Community district ratio grid with two-decimal labels



7.2 Project 2: Visualize baseball player batting averages with filters and rank

Procedure: The baseball-player-stats.xlsx dataset was added to the same Tableau Public workbook through the Microsoft Excel connector, and the 2012 Team Player Stats sheet was dragged onto the data source canvas. In a new worksheet, the Name dimension was placed on the Rows shelf and the BA measure was placed on the Columns shelf, producing a horizontal bar chart that was subsequently sorted in descending order. The AB measure was then dragged onto the Filters shelf, the At least option was selected, and a threshold of 100 at-bats was

applied in order to remove players with insufficient playing time. A quick filter for AB was subsequently exposed through the Show Filter command, and the mark type was changed from Automatic to Circle in the Marks card. A calculated field named Rank was then created using RANK(SUM([BA])), its table calculation was configured to compute along Name, the field was converted to Discrete, and it was positioned to the left of Name on the Rows shelf alongside the BA measure.

Result/Observation: The initial bar chart displayed batting averages for all 45 players and revealed that approximately half of the roster held a value of 0.00, with a null indicator confirming that 16 players had no recorded at-bats. After the AB filter was applied with a threshold of 100, the view was reduced to 11 qualified batters, and Ichiro Suzuki emerged as the leader with a batting average of 0.322. This transformation demonstrates how filtering reduces visual clutter by removing data points that do not contribute meaningful signal, a principle that Knaflic identifies in *Storytelling with Data* as essential for directing audience attention. The change from bar to circle marks further illustrates Munzner's argument in *Visualization Analysis and Design* that visual encoding should be matched to the analytical task; circles better support rank-based scanning than bars when the magnitude differences are small. The Rank field, applied as a table calculation, corresponds to the aggregate-level technique discussed by Loth in *Visual Analytics with Tableau*, where ranking is treated as a derived attribute rather than a raw data field.

Screenshots:

Figure 7.5: 2012 Team Player Stats dataset loaded into the data source page

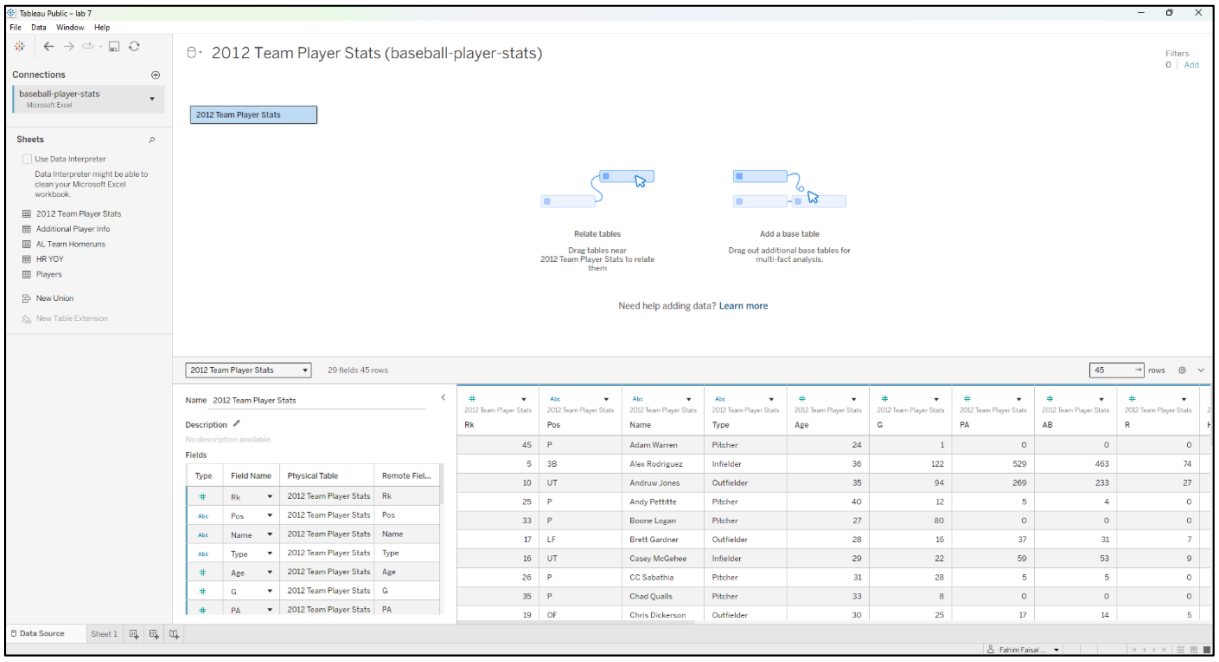


Figure 7.6: Bar chart of batting averages for all 45 players

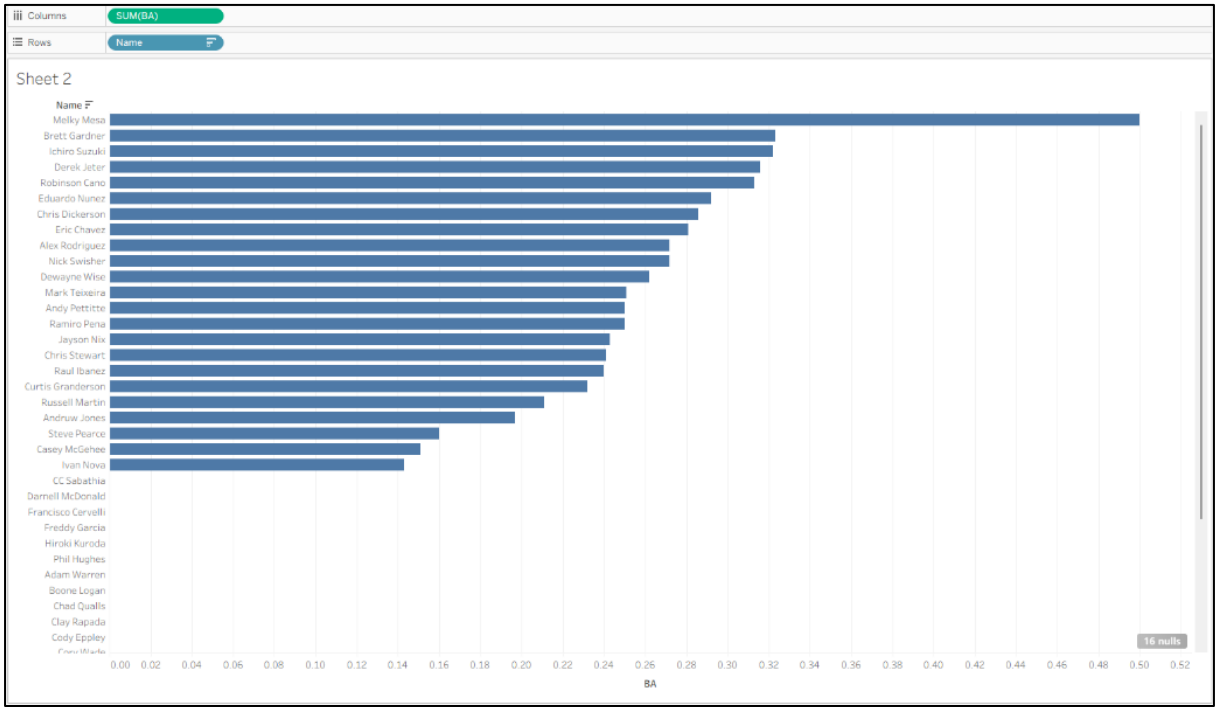


Figure 7.7: Batting averages after applying the AB ≥ 100 filter

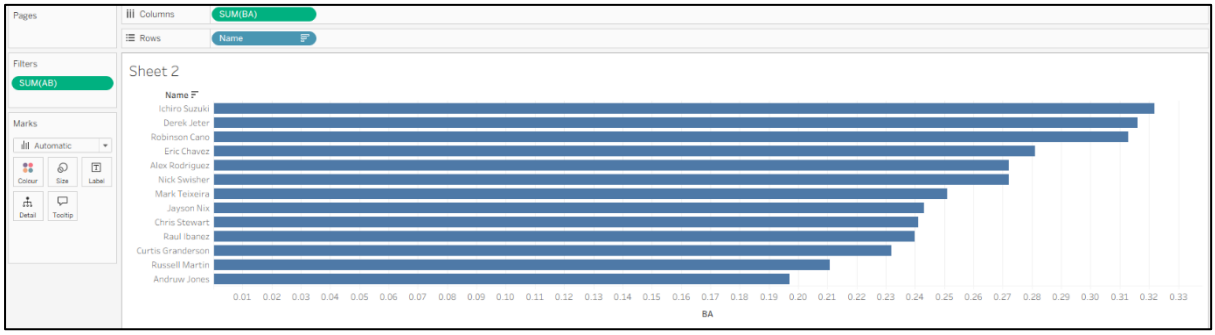


Figure 7.8: Dot chart with the AB quick filter slider

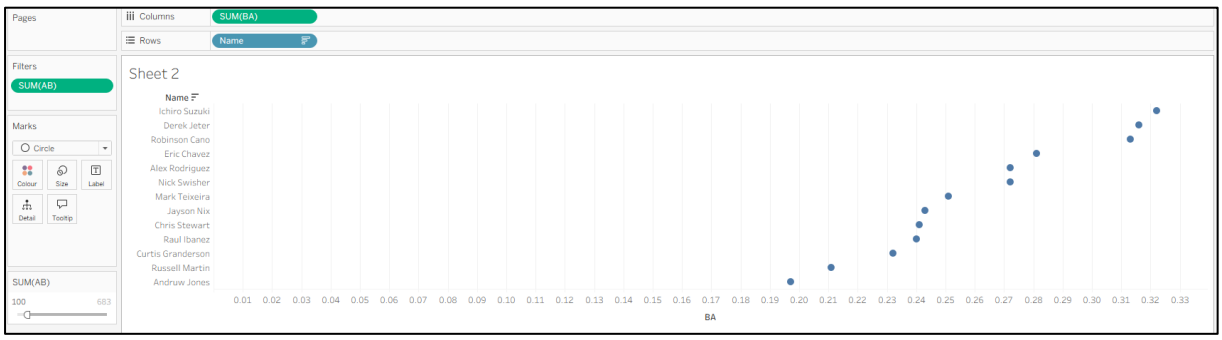
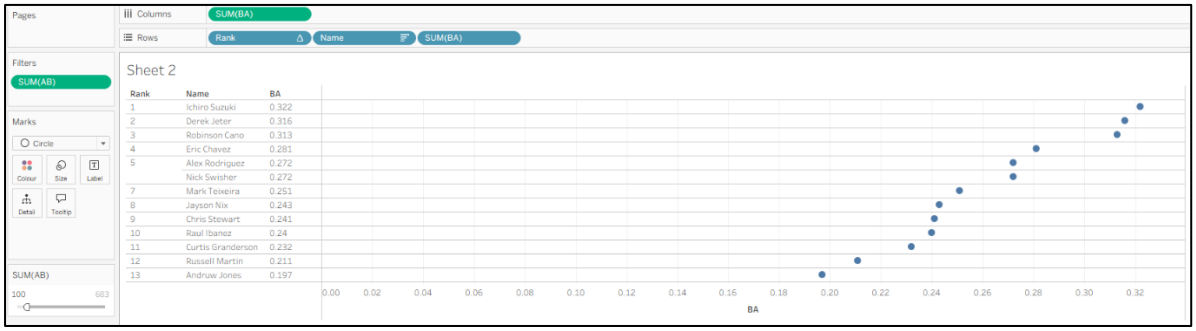


Figure 7.9: Ranked dot chart with Rank, Name, and BA on the Rows shelf



7.3 Project 3: Visualize home runs with team totals and percent of total

Procedure: A new worksheet was created within the same Tableau Public workbook and the 2012 Team Player Stats data source was reused. The Name dimension was placed on the Rows shelf, the HR measure was placed on the Columns shelf, and the resulting bar chart was sorted in descending order. The HR measure was then dragged onto the Filters shelf and the At least option was selected with a value of 1 in order to restrict the view to players who had hit at least one home run. The Rank calculated field was positioned to the left of Name on the Rows shelf, its table calculation was set to compute along Name, and the formula was changed to RANK(SUM([HR])) so that ranking reflected home run totals rather than batting averages. The Column Grand Totals option was activated from the Analytics pane, and the HR measure was dragged onto the Label shelf of the Marks card. Finally, the Percent of Total quick table calculation was applied to SUM(HR) and the number format was changed to percentage with two decimal places.

Result/Observation: The final view displayed 18 players who had recorded at least one home run, with Curtis Granderson leading at 43 home runs and Robinson Cano ranking second at 33. The grand total bar at the bottom of the chart confirmed that the team accumulated 245 home runs across the season, and the percent-of-total calculation showed that Granderson contributed 17.55% of this total while Cano contributed 13.47%. This combination of absolute and relative measures illustrates the analytical distinction that Munzner draws in *Visualization Analysis and Design* between magnitude and proportion as separate derived-data attributes. The use of percentage formatting aligns with the recommendation in Loth's *Visual Analytics with Tableau* that ratio-based measures be expressed in relative units when the total serves as a meaningful reference point. Grand totals and percent-of-total calculations also demonstrate how Tableau applies aggregation at the level of the view rather than the underlying rows, producing a representation that is both precise and interpretable.

Screenshots:

Figure 7.10: Home run bar chart for players with at least one home run

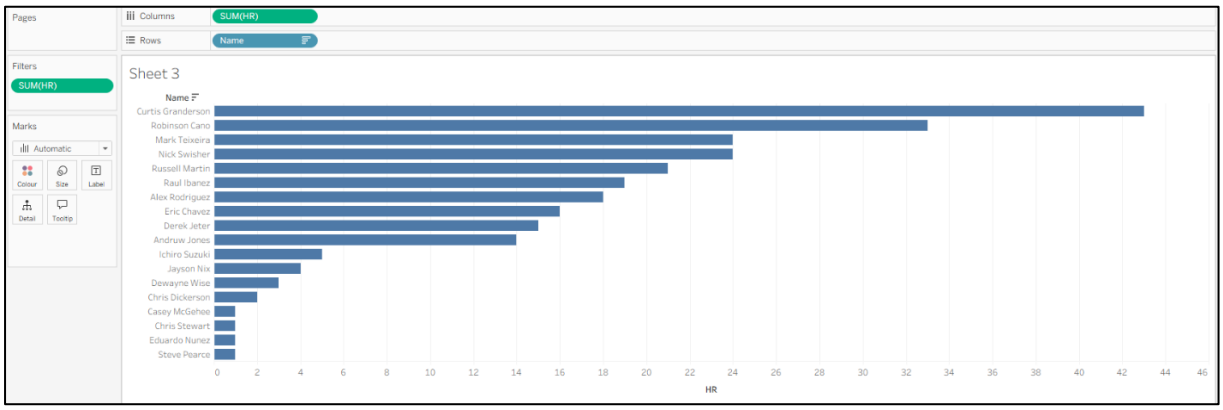


Figure 7.11: Ranked home run chart with the corrected RANK(SUM([HR])) formula

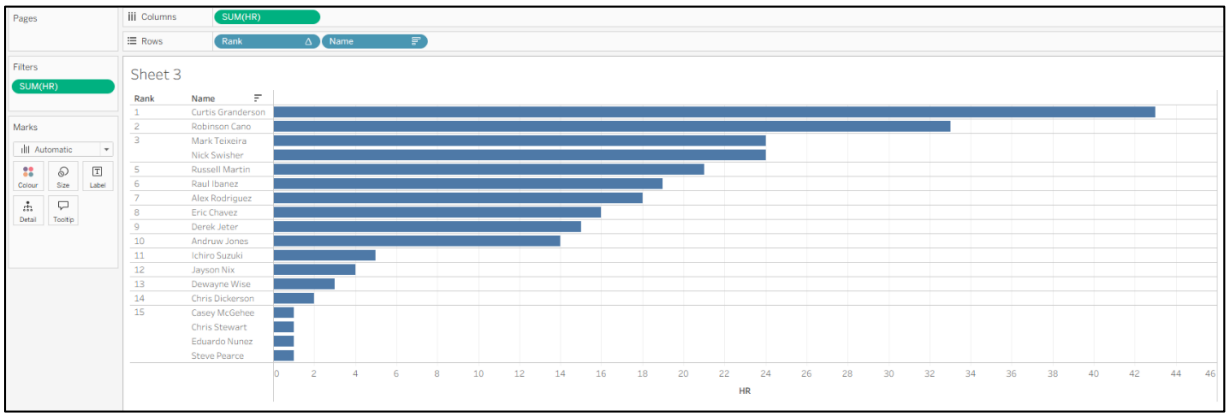


Figure 7.12: Home run chart with Column Grand Total and HR labels

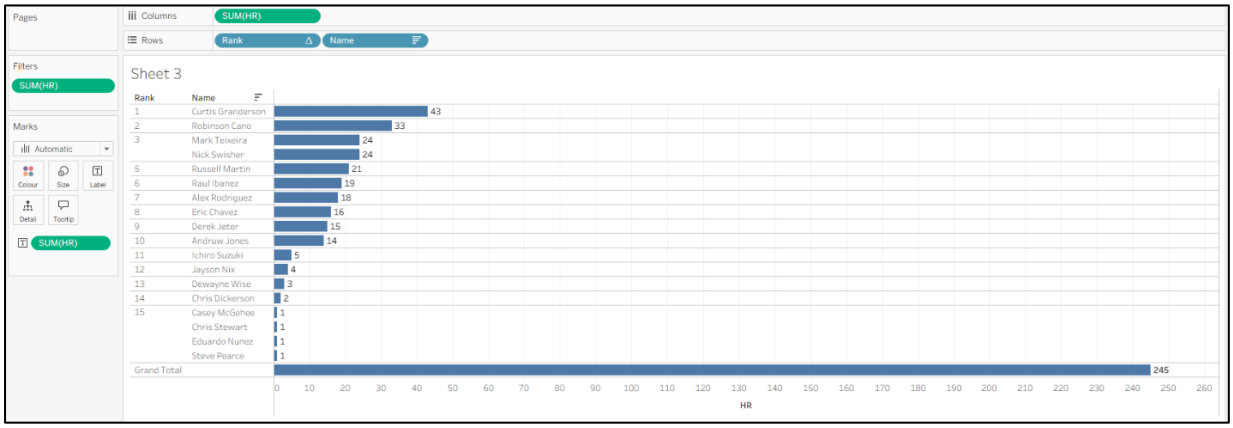
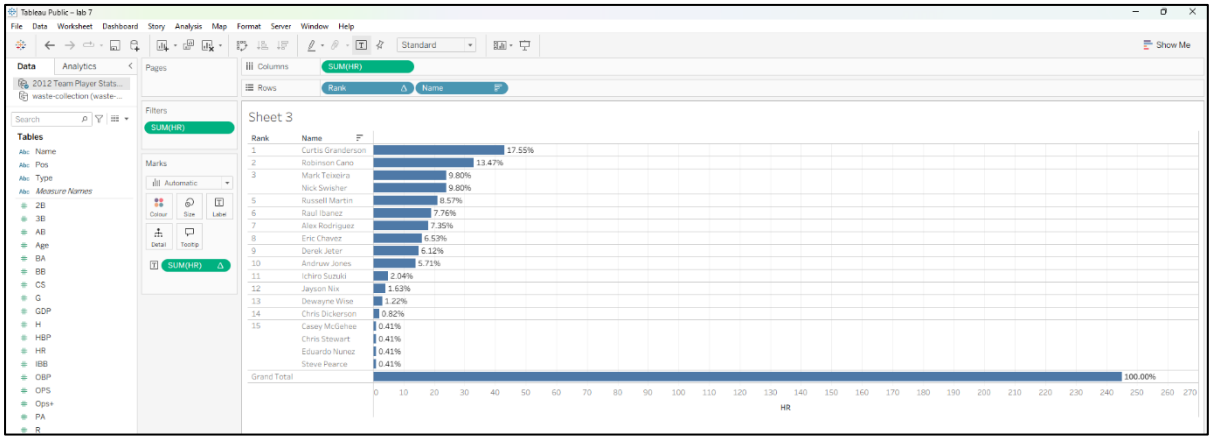


Figure 7.13: Home run chart showing percent of total contribution per player



Chapter 8: Complex data visualizations (3)

This chapter extends the analytical capabilities demonstrated in previous chapters by applying Tableau's built-in analytics tools to the order-information.xls dataset. The exercises examine trend modelling, time-series forecasting, unsupervised clustering, and interactive parameter controls, each of which supports analytical questions that cannot be answered through descriptive visualizations alone. All work was performed in Tableau Public on Windows, and the dataset comprises 9,994 order records with 21 fields including Sales, Profit, Order Date, Customer ID, Customer Name, Region, and Sub-Category.

8.1 Project 1: Visualize trend

Procedure: The order-information.xls dataset was loaded into Tableau Public through the Microsoft Excel connector, and the Orders table was dragged onto the data model canvas. After selecting the Sheet 1 tab, the Sales measure was dragged from the Measures section of the Data pane onto the Rows shelf, and the Order Date dimension was dragged onto the Columns shelf. The resulting YEAR(Order Date) pill was clicked, the Month submenu item was selected, and the Continuous option was applied so that the axis rendered a continuous month-level time series. From the Analytics pane, the Trend Line item was dragged onto the view, and the Polynomial model was selected from the floating Add a Trend Line panel. The trend line was right-clicked, Edit All Trend Lines... was selected, the Degree was changed from 2 to 3, and the Show confidence bands option was left unchecked. Finally, Describe Trend Line... and Describe Trend Model... were invoked from the same context menu, and the resulting statistics were captured as screenshots.

Result/Observation: The line chart revealed substantial month-to-month volatility in sales, with prominent peaks near November of each year and a general upward trajectory that became more pronounced after November 2013. The polynomial trend line of degree three confirmed this pattern, rising from approximately 15,000 at the beginning of the series to roughly 105,000 by the end. The Describe Trend Model dialog reported an R-squared value of 0.334798 and a p-value of 0.0004131, indicating that the polynomial model explains approximately one third of the variance in monthly sales and is statistically significant at the conventional 0.05 threshold. As Munzner emphasises in *Visualization Analysis and Design*, the selection of an aggregation and a trend model are both derived-data decisions that determine which patterns become visible; here, the third-degree polynomial exposes the accelerating revenue growth that a linear trend line would have obscured. The result also demonstrates that Tableau applies trend estimation to the current level of aggregation rather than to individual rows, so the number of modelled observations was reported as 48 monthly data points rather than 9,994 transactions.

Screenshots:

Figure 8.1: Orders data source loaded in Tableau Public

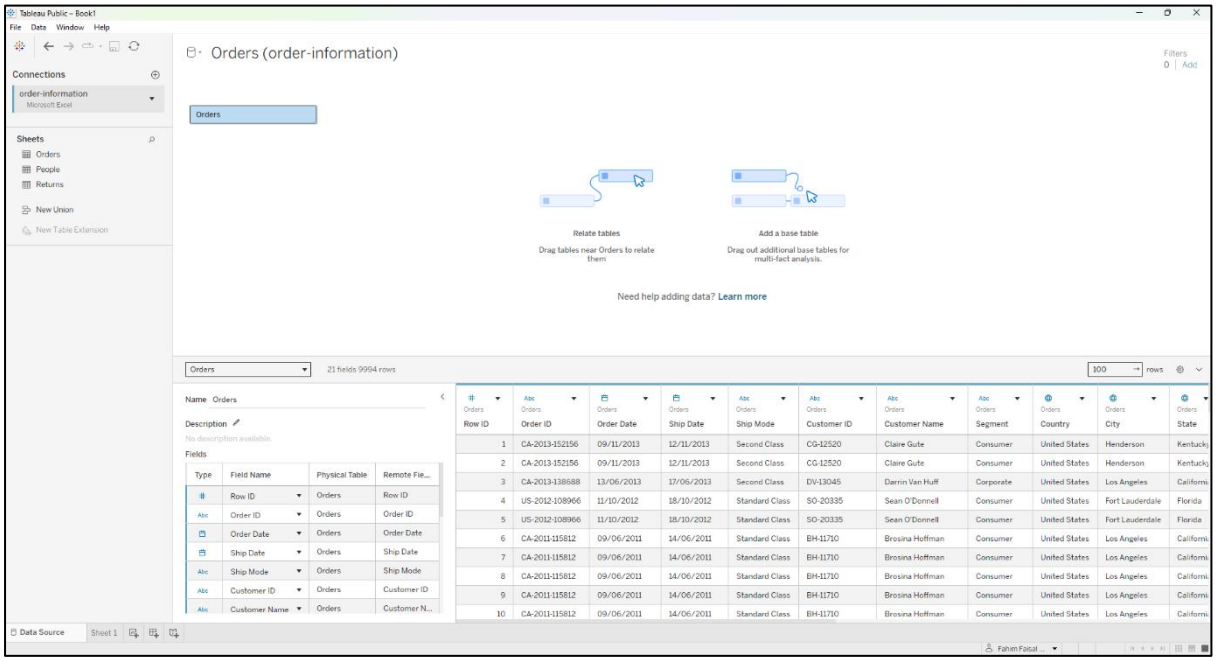


Figure 8.2: Line chart of sales by month of order date

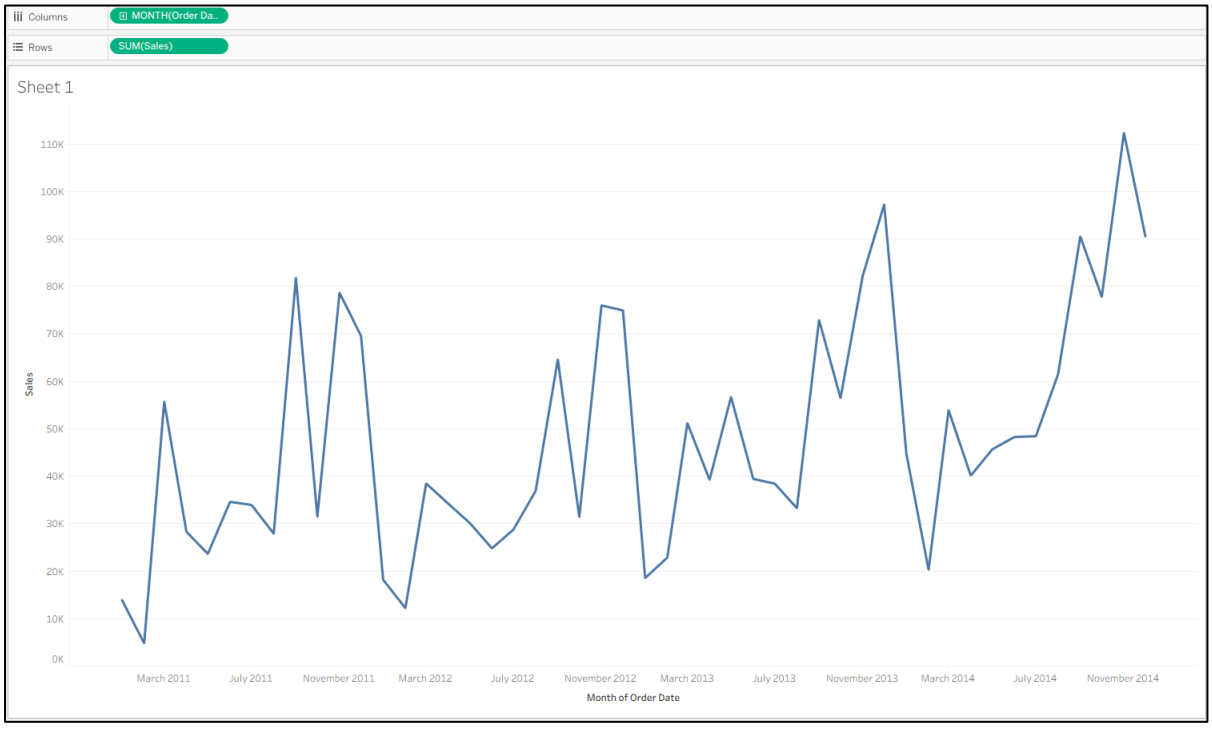


Figure 8.3: Polynomial trend line added to the sales line chart

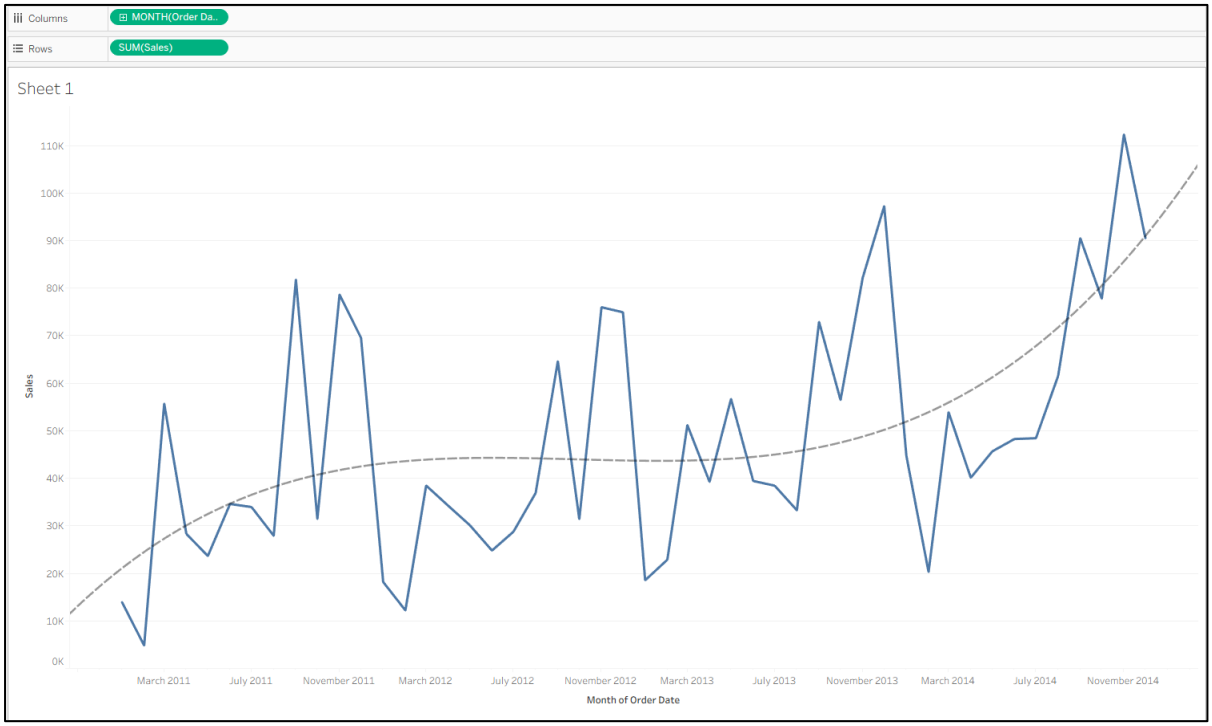


Figure 8.4: Trend Lines Options dialog configured with a degree three polynomial

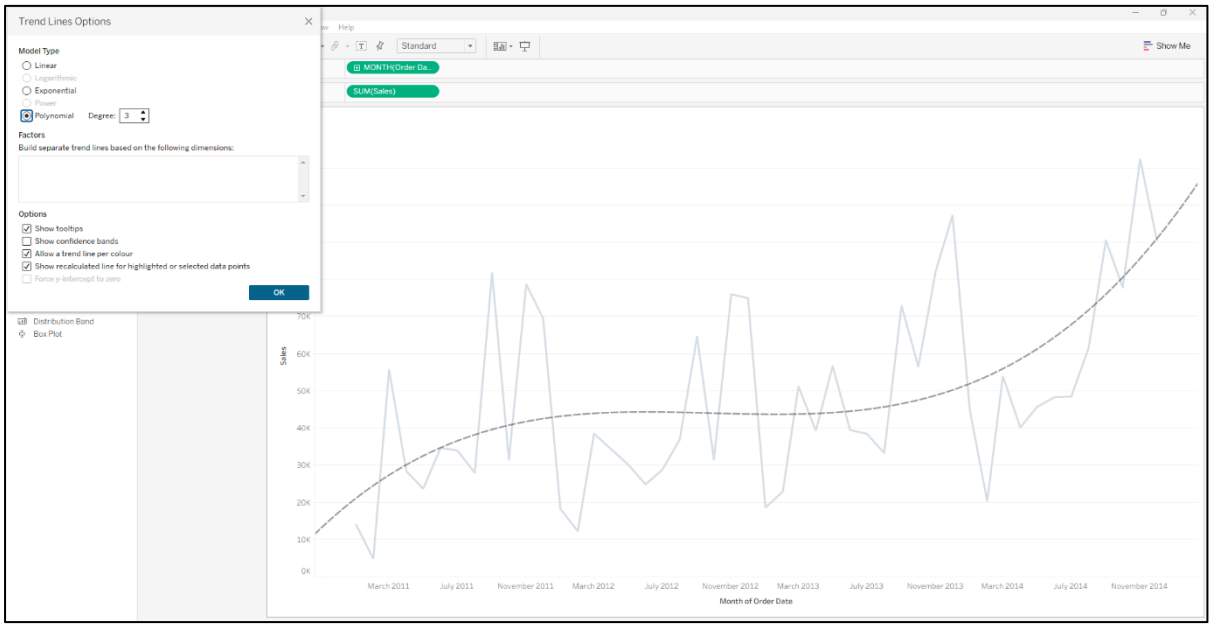


Figure 8.5: Describe Trend Line dialog showing the regression equation and p-value

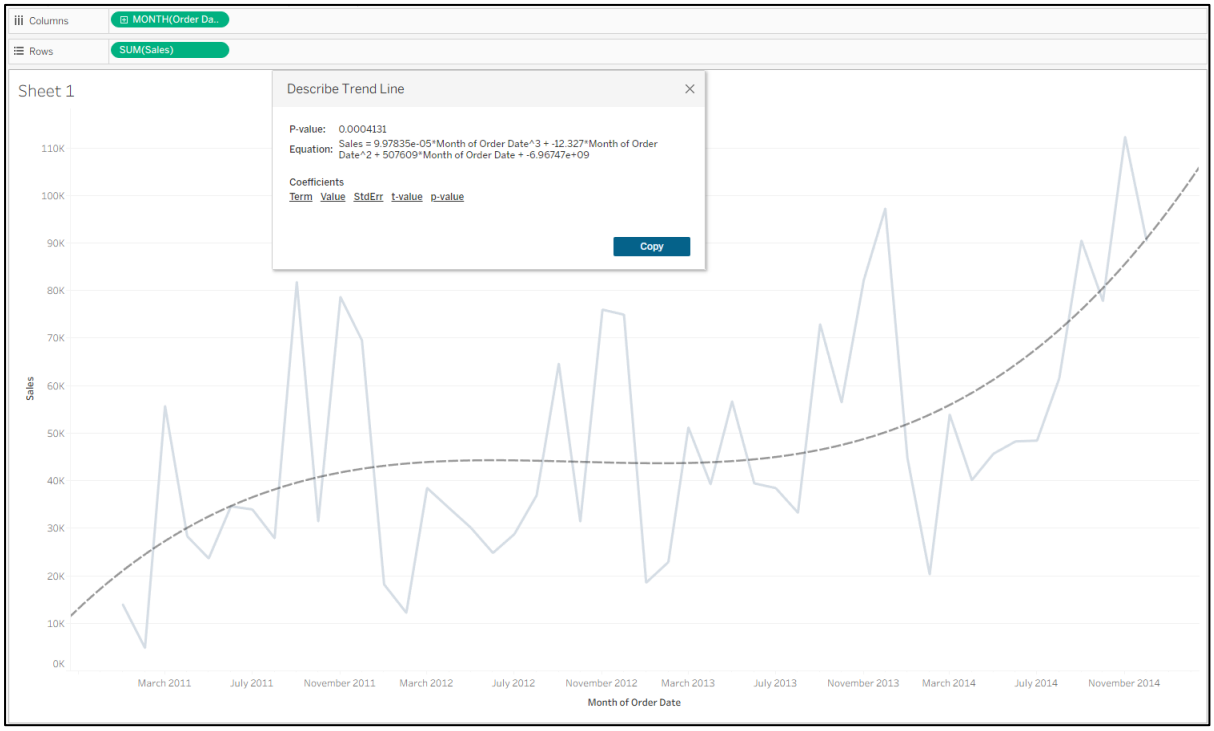


Figure 8.6: Describe Trend Model dialog showing the full statistical summary



8.2 Project 2: Visualize forecasts

Procedure: A new worksheet was created within the same workbook, and the Sales measure was again dragged onto the Rows shelf while the Order Date dimension was dragged onto the Columns shelf. The default YEAR(Order Date) pill was converted to a continuous MONTH(Order Date) field to produce a line chart of monthly sales over time. The Forecast item was then dragged from the Model section of the Analytics pane onto the view, and Tableau automatically generated an extended time series with a prediction interval. The chart was right-clicked, the Forecast submenu was expanded, and Forecast Options... was selected to open the configuration dialog. The default settings were verified, including a forecast length of thirteen months, an Automatic (Months) aggregation, an Ignore last value of one month, a 95% prediction interval, and an automatic exponential smoothing model.

Result/Observation: The forecast extended the line chart from November 2014 to December 2015 and displayed the projected sales values within a shaded 95% prediction interval. The uncertainty band widened substantially towards the end of the forecast horizon, which reflects the compounding variance that arises as the model extrapolates further from the observed data. The Forecast Options dialog reported that the underlying model was automatically selected and that Tableau was searching for seasonal patterns at a twelve-month periodicity. As Loth discusses in *Visual Analytics with Tableau*, Tableau's forecasting feature applies exponential smoothing with trend and seasonality components, which is why the projected line continued the upward trajectory established in the historical data rather than reverting to a horizontal mean. This behaviour illustrates a key tension identified by Knaflic in *Storytelling with Data*: forecasts are persuasive because they extend a narrative into the future, but they must be presented with their uncertainty intervals so that the audience does not mistake a probabilistic estimate for a certainty.

Screenshots:

Figure 8.7: Line chart of monthly sales prior to forecasting

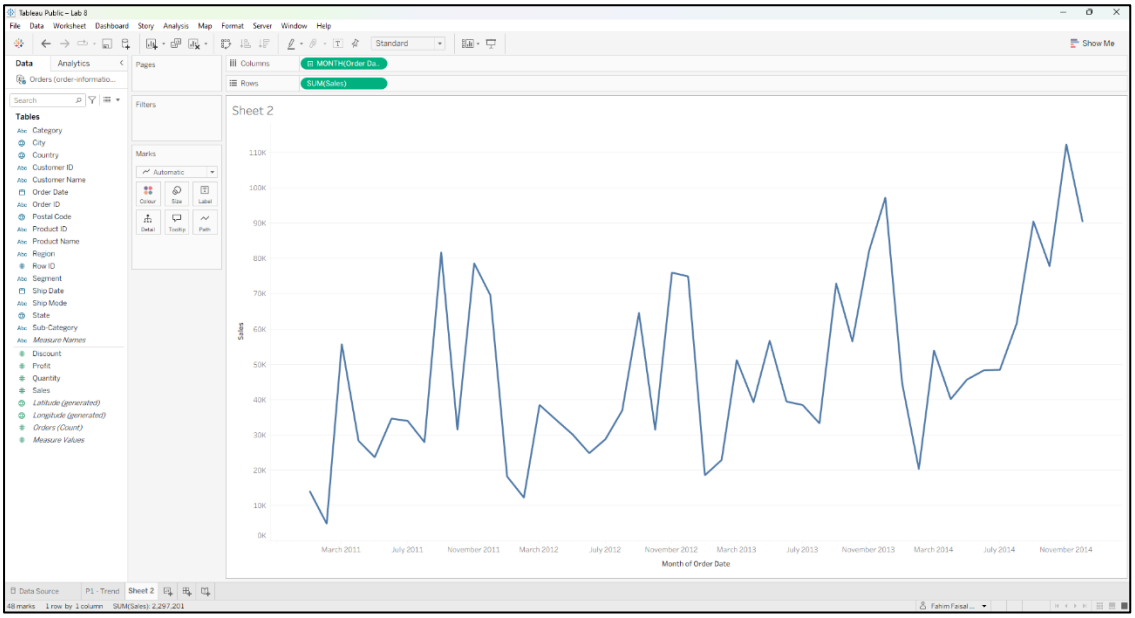


Figure 8.8: Forecast line and prediction interval added to the sales chart

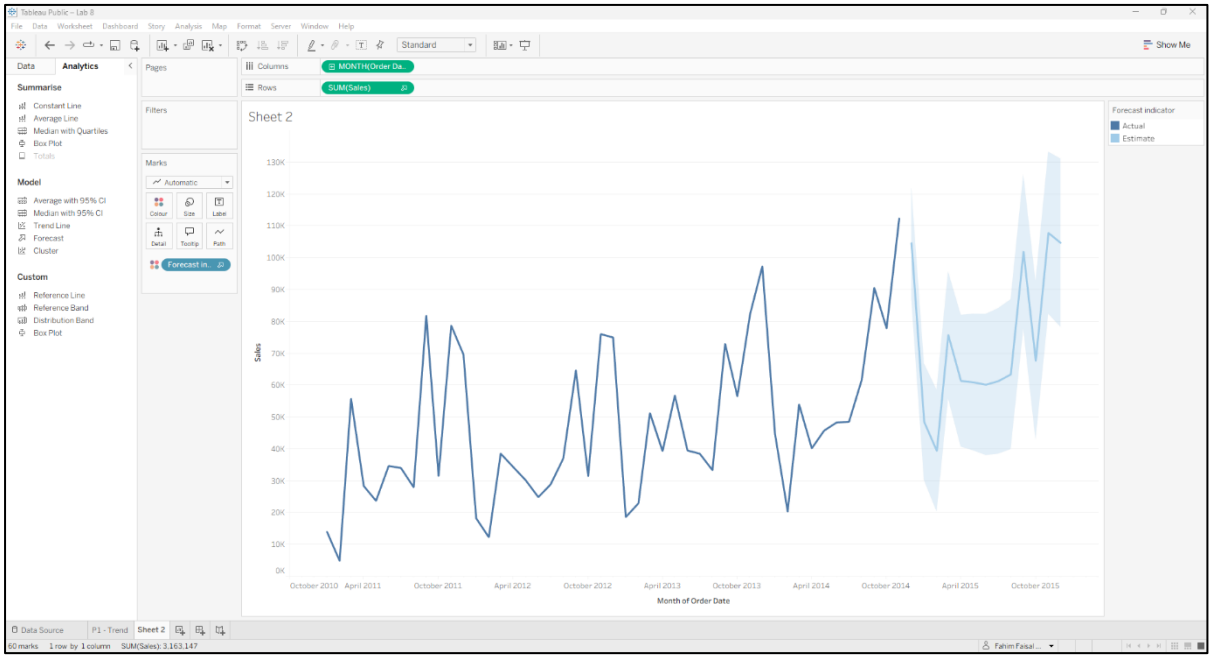
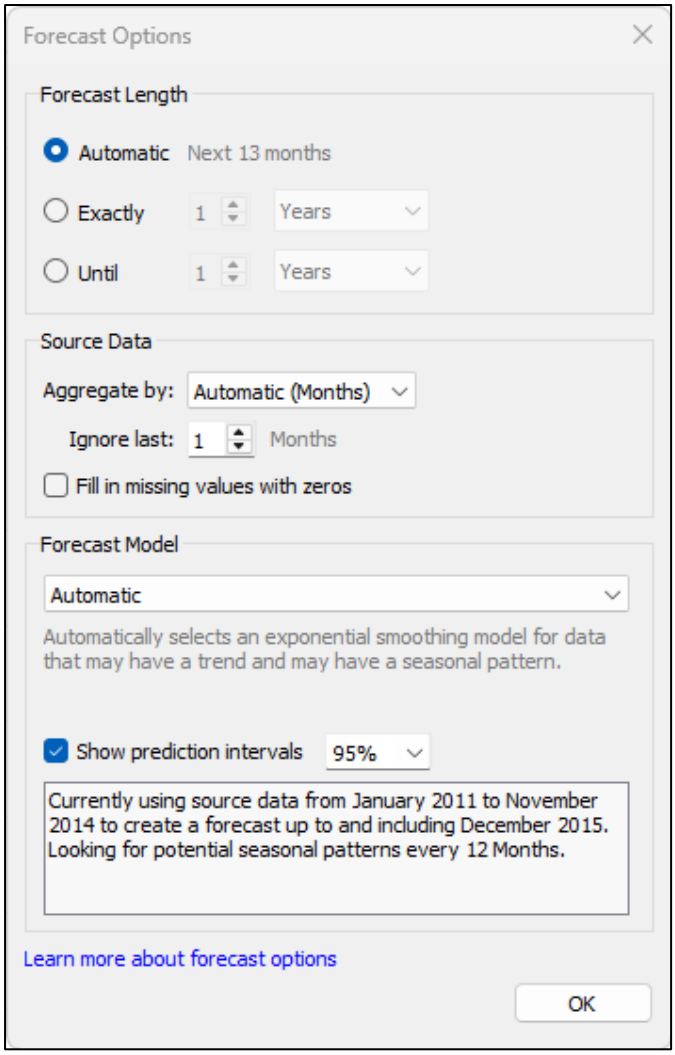


Figure 8.9: Forecast Options dialog with automatic settings and a 95% prediction interval



8.3 Project 3: Visualize cluster

Procedure: A third worksheet was created, and the Profit measure was dragged onto the Rows shelf while the Sales measure was dragged onto the Columns shelf. To produce one mark per customer, the Customer ID dimension was dragged onto the Detail button of the Marks card. The Cluster item was then dragged from the Analytics pane onto the scatter plot, and Tableau automatically generated two clusters by default while placing a new Clusters dimension on the Colour button of the Marks card. The Clusters dialog box was used to change the Number of Clusters field from Automatic to 4, at which point the marks were divided into four colour-coded segments. To preserve the clustering result as a reusable field, the Clusters pill was dragged from the Marks card into the Dimensions section of the Data pane, and the default name Customer ID (clusters) was accepted. A new worksheet was then created, Sales was dragged onto the Rows shelf, Region was dragged onto the Columns shelf, and Customer ID (clusters) was dragged onto the Columns shelf to the right of Region, producing a nested view of sales by region and cluster.

Result/Observation: The scatter plot revealed that the majority of customers were concentrated in a dense band near the origin, where both sales and profit were relatively low, while a small number of customers generated substantially higher sales and profit and appeared as isolated outliers in the upper-right region of the plot. After applying the k-means clustering algorithm with four clusters, the segments arranged themselves into a clear ordinal pattern: a large low-sales, low-profit cluster; a narrow cluster with negative profit and low sales; a medium-sales, medium-profit cluster; and a small high-value cluster. The final worksheet comparing sales by region and cluster showed that the medium-value cluster contributed the largest total sales in every region, with the West region generating the highest overall sales of approximately 330,000, while the high-value cluster contributed the smallest total sales because it contained only a few customers. As Munzner explains in *Visualization Analysis and Design*, clustering is a form of derived-data transformation that converts a two-dimensional scatter of points into a categorical attribute, which can then be used as a visual channel for further analysis. This project demonstrates that unsupervised clustering in Tableau is not an end in itself but rather a starting point for downstream questions about how segments are distributed across other dimensions of the data.

Screenshots:

Figure 8.10: Scatter plot of profit against sales with one mark per customer

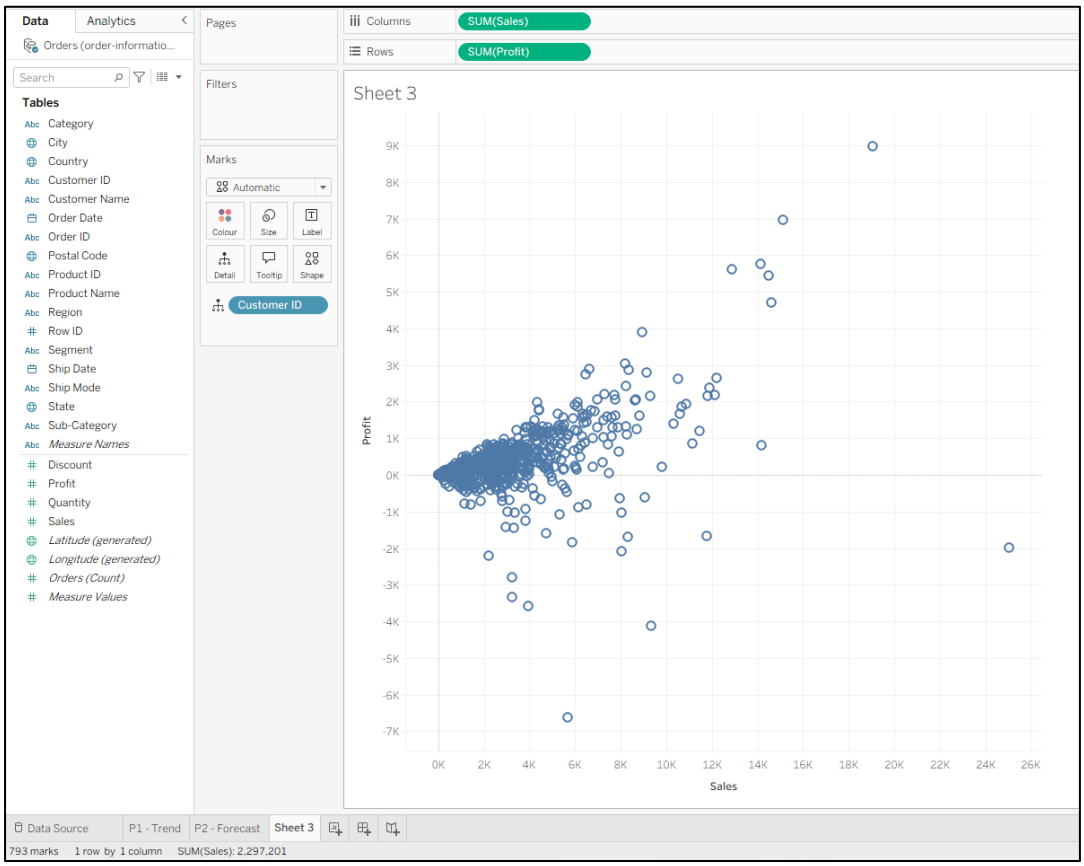


Figure 8.11: Initial two-cluster result generated automatically by Tableau



Figure 8.12: Four-cluster result after setting the number of clusters to four



Figure 8.13: Customer ID (clusters) field saved to the Data pane

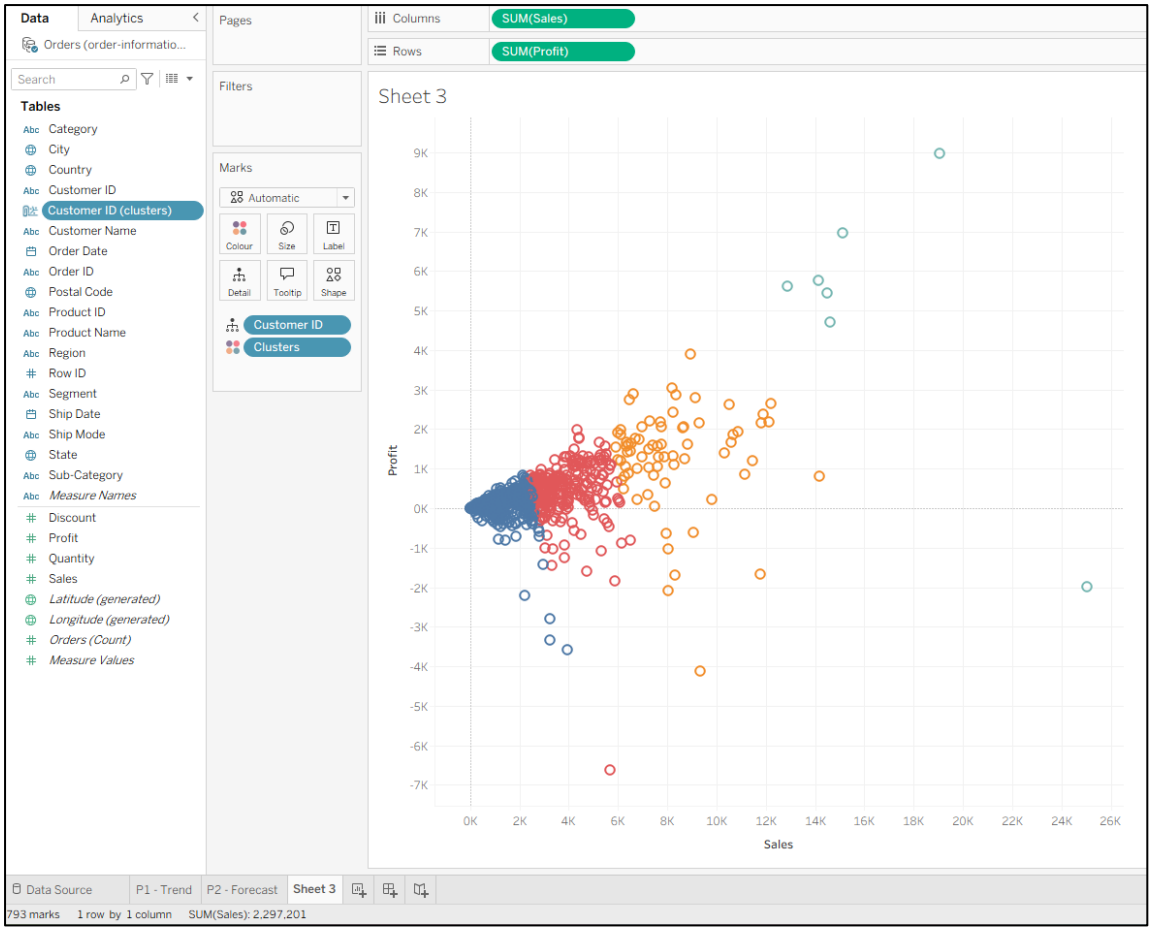
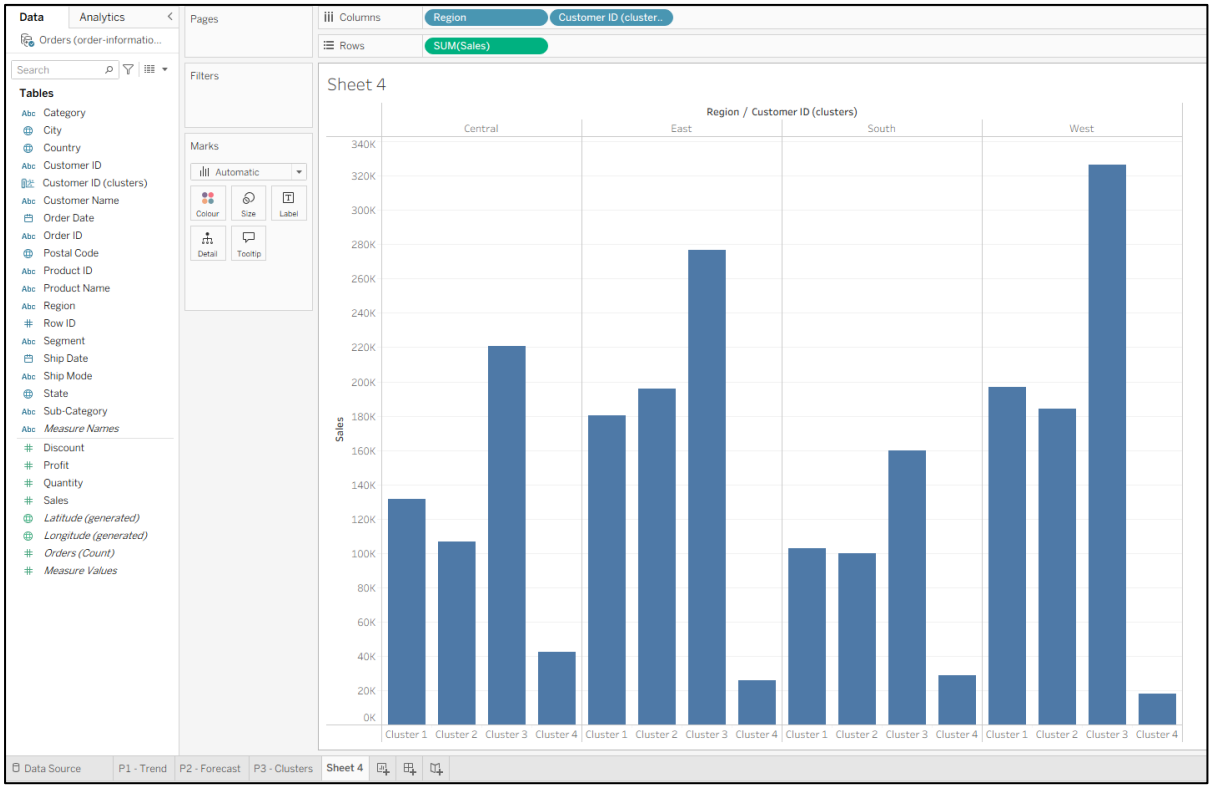


Figure 8.14: Sales by region and customer cluster in a nested bar chart



8.4 Project 4: Use parameter control

Procedure: A new worksheet was created, and a parameter named Factor was defined through the Create Parameter command in the Data pane menu. The parameter was configured with a Float data type, a current value of 1.1, a Range of allowable values between 0.1 and 2, and a step size of 0.1. The parameter control was displayed in the view by right-clicking the Factor entry in the Parameters section and selecting Show Parameter. A calculated field named Adjusted Sales was then created from the Factor parameter with the formula SUM([Sales]) * [Factor], and this field was dragged onto the Columns shelf while the Sub-Category dimension was dragged onto the Rows shelf. The bars were sorted in descending order by Adjusted Sales. A second parameter named Search was created with a String data type, a blank current value, and All as the allowable values. A calculated field named Match was created from the Search parameter with the formula CONTAINS(LOWER([Customer Name]), LOWER([Search])), and this field was dragged onto the Filters card, where only the True value was selected. A final worksheet was created with Sales on the Columns shelf and Customer Name on the Rows shelf, and the Match: True filter was copied to this sheet. The Search parameter control was displayed, and the text chris was typed into it.

Result/Observation: The Adjusted Sales bar chart displayed the total sales for each product sub-category multiplied by the current value of the Factor parameter, and changing the slider from 1.1 to 2 visibly doubled the length of every bar without requiring any manual recalculation, confirming that Tableau evaluates the parameter against the aggregation on the fly. The Search parameter demonstrated a complementary pattern: when the string chris was typed into the

search box, the chart was reduced from approximately 1,048 customer records to fifteen customers whose names contained that substring, including Christopher Conant, Christopher Martinez, and Christina Anderson. Because the Match formula applied the LOWER function to both the customer name and the search string, the filter behaved case-insensitively. As Loth discusses in *Visual Analytics with Tableau*, parameters are one of the primary mechanisms by which Tableau supports what-if analysis and interactive exploration, because they decouple the value that drives a calculation from the data source itself. This project illustrates that a well-designed parameter control allows an analyst to publish a single workbook that answers a family of related questions, which is a central principle of the dashboard design approaches discussed in the lecture material on storytelling with data.

Screenshots:

Figure 8.15: Create Parameter dialog for the Factor parameter

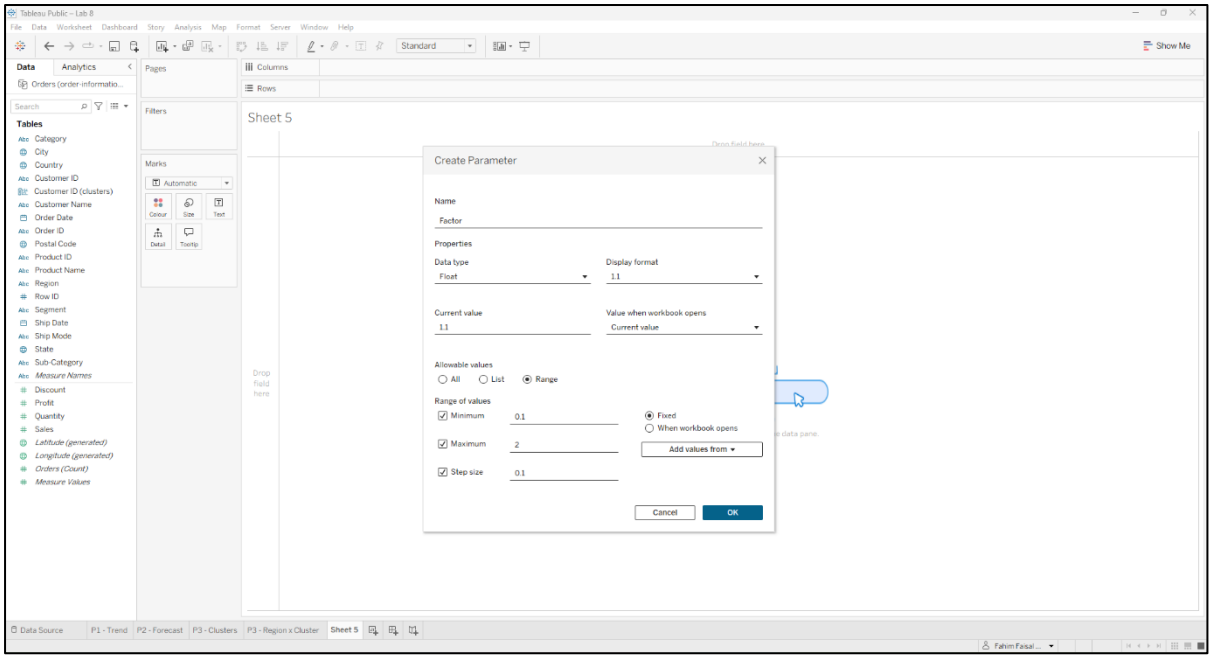


Figure 8.16: Factor parameter appearing in the Parameters section of the Data pane

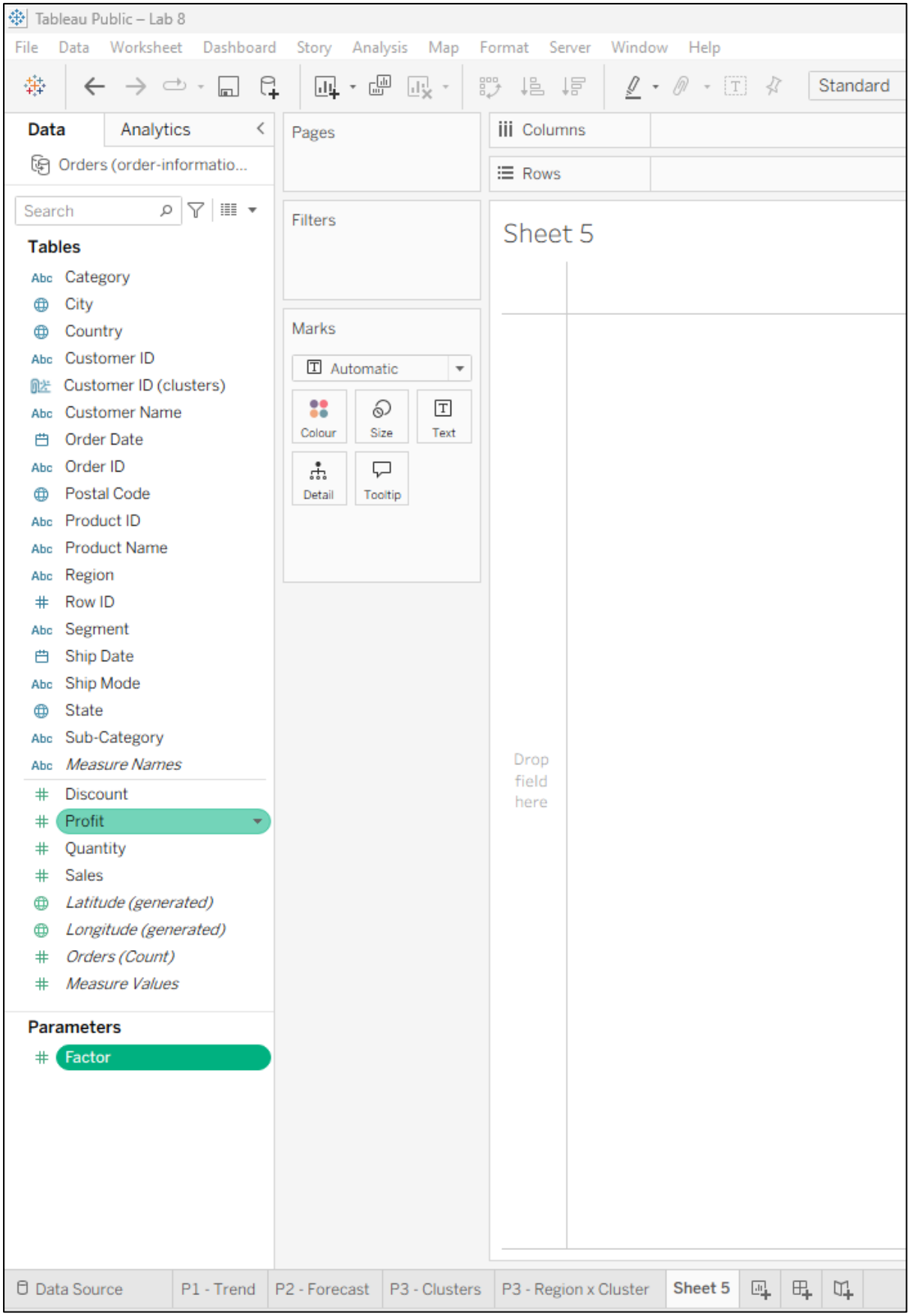


Figure 8.17: Factor parameter control displayed on the worksheet

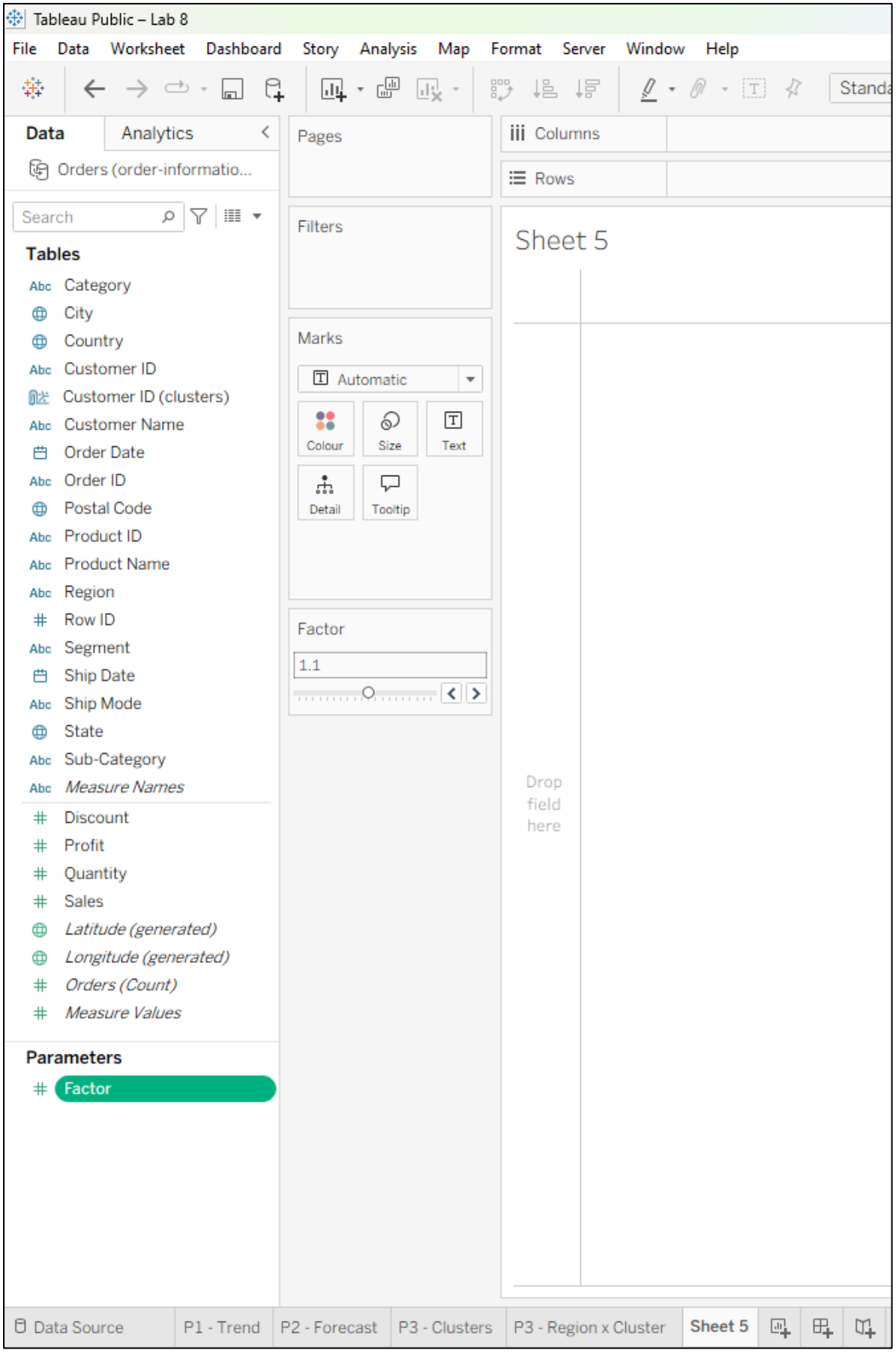


Figure 8.18: Adjusted Sales calculated field using the Factor parameter

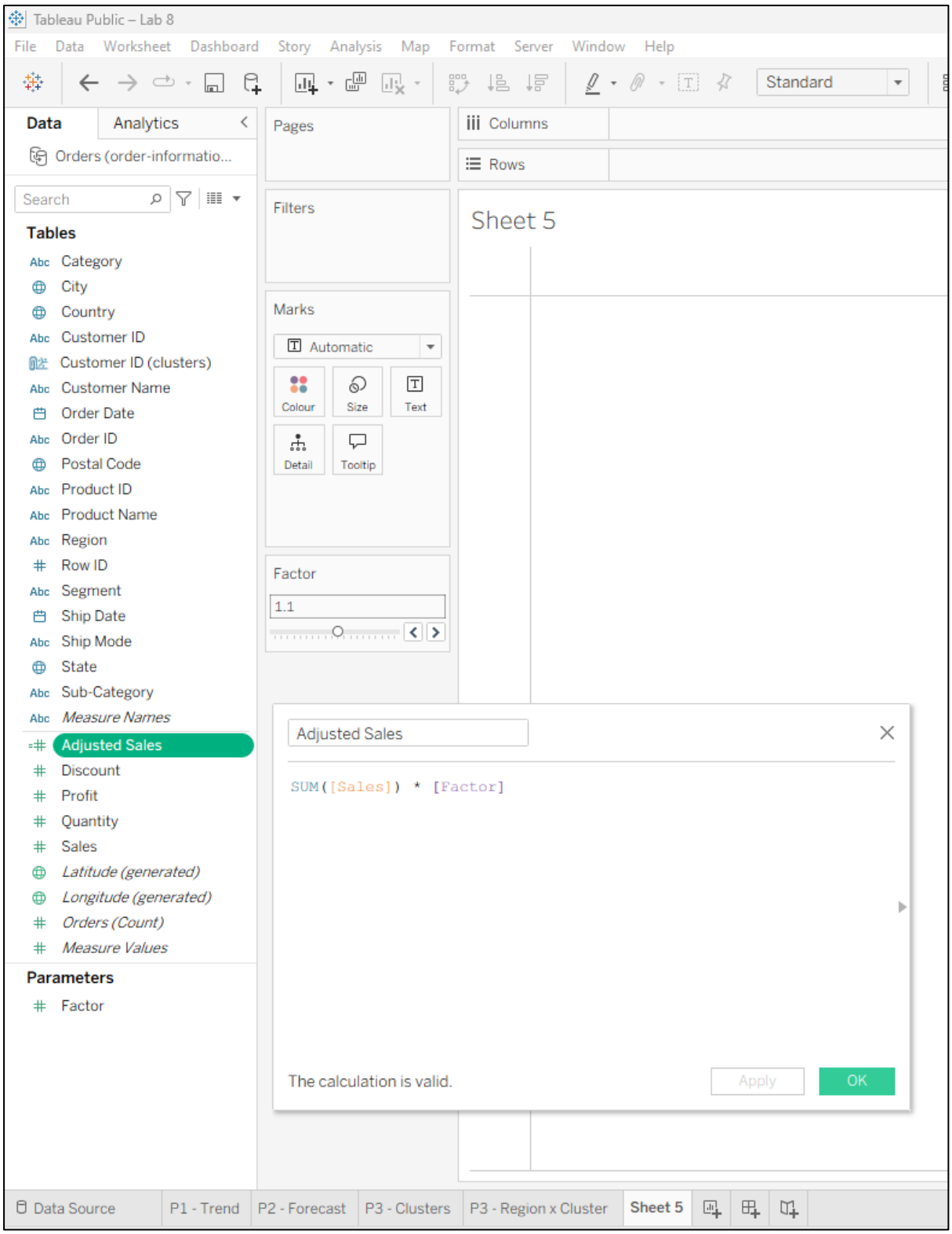


Figure 8.19: Bar chart of Adjusted Sales by sub-category at Factor = 1.1

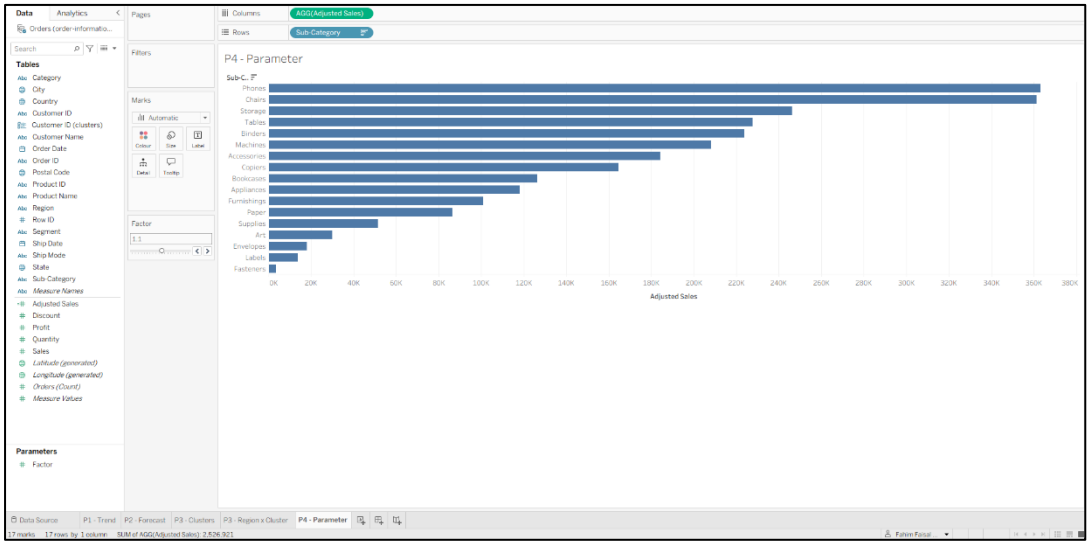


Figure 8.20: Create Parameter dialog for the Search parameter

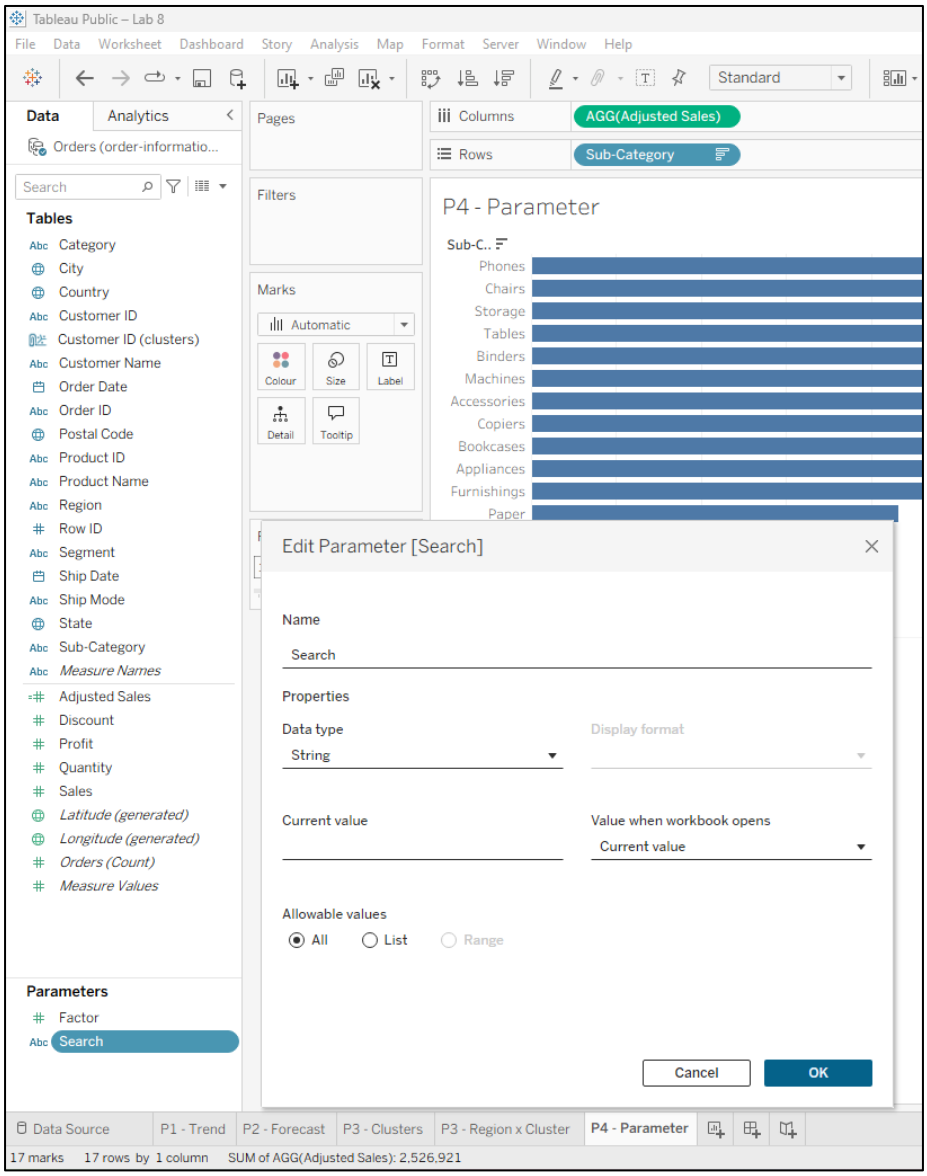


Figure 8.21: Match calculated field using the CONTAINS and LOWER functions

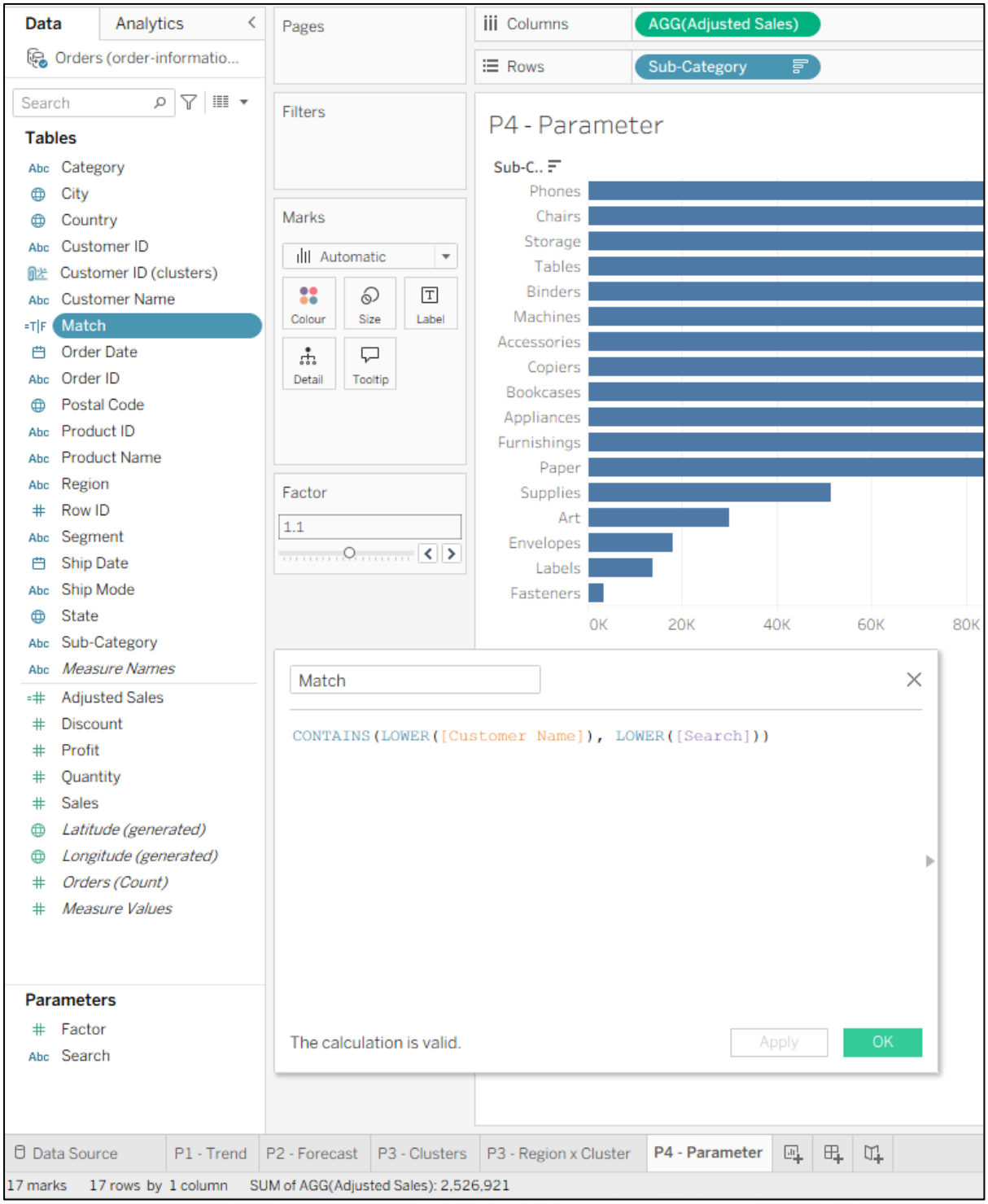


Figure 8.22: Match filter applied to the P4 - Parameter worksheet

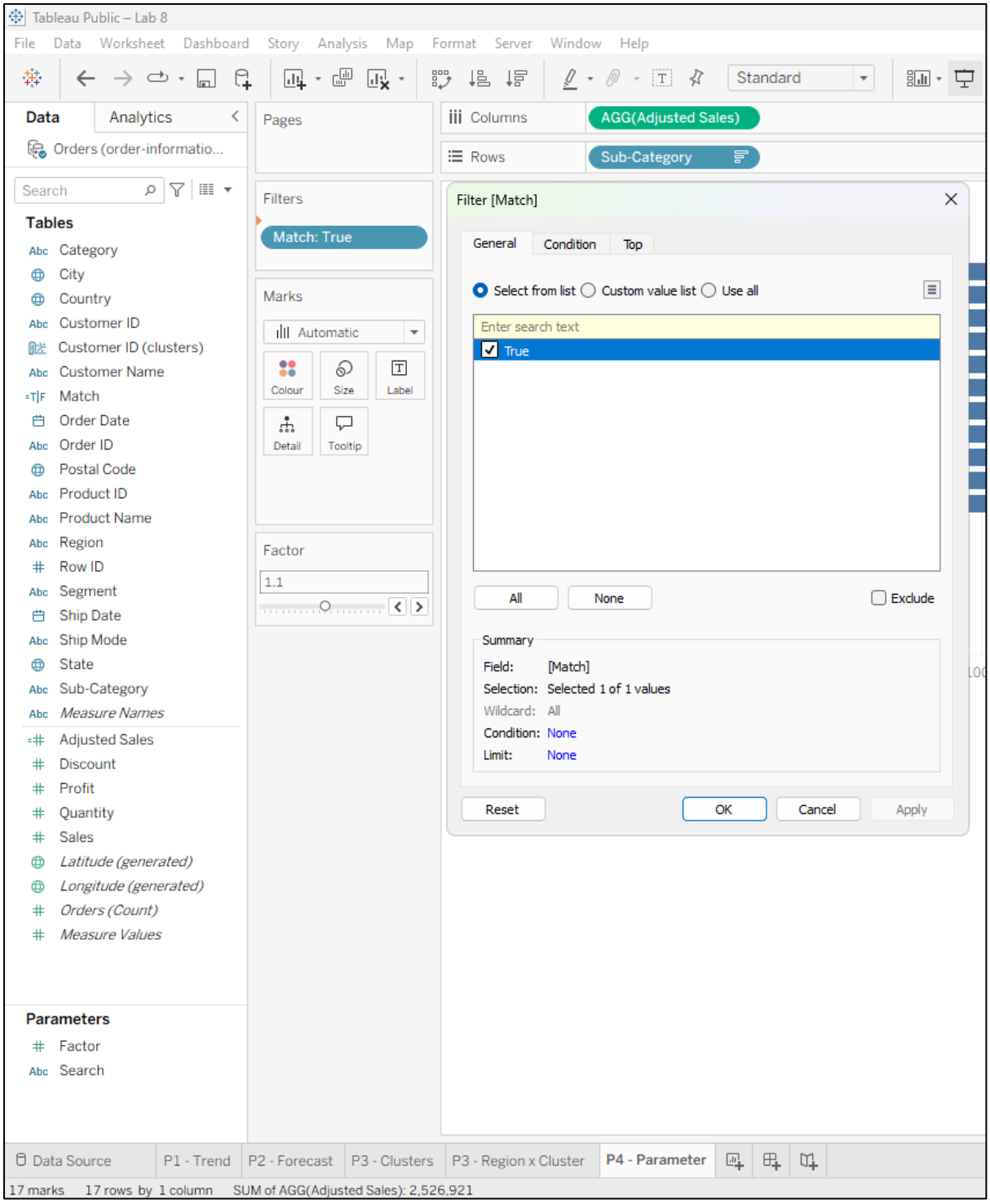


Figure 8.23: Sales by customer name before applying the search filter

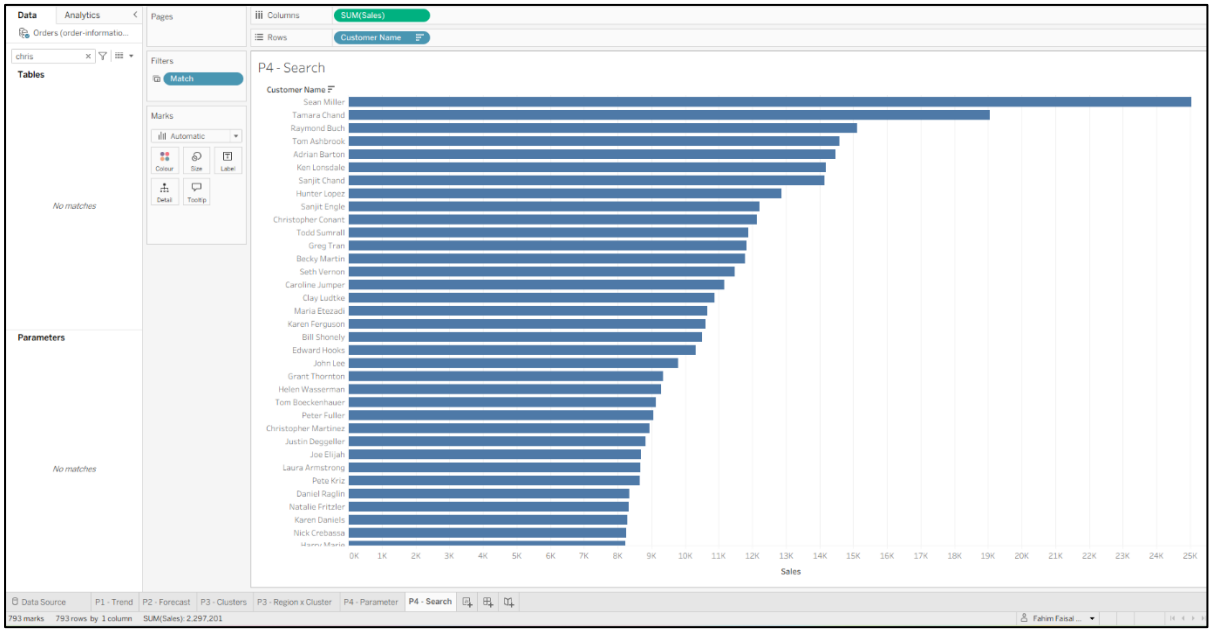


Figure 8.24: Sales by customer name filtered by the search string chris

